Assignment_3

Nishant Khandhar (A20581012)

2025-10-14

Assignment 3

Problems from R for Data Science book:

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Section 9.3.1: Problem 3

Q) What does the se argument to geom_smooth() do?

Ans: It displays the confidence interval around the smooth lin. You can remove this with se = FALSE.

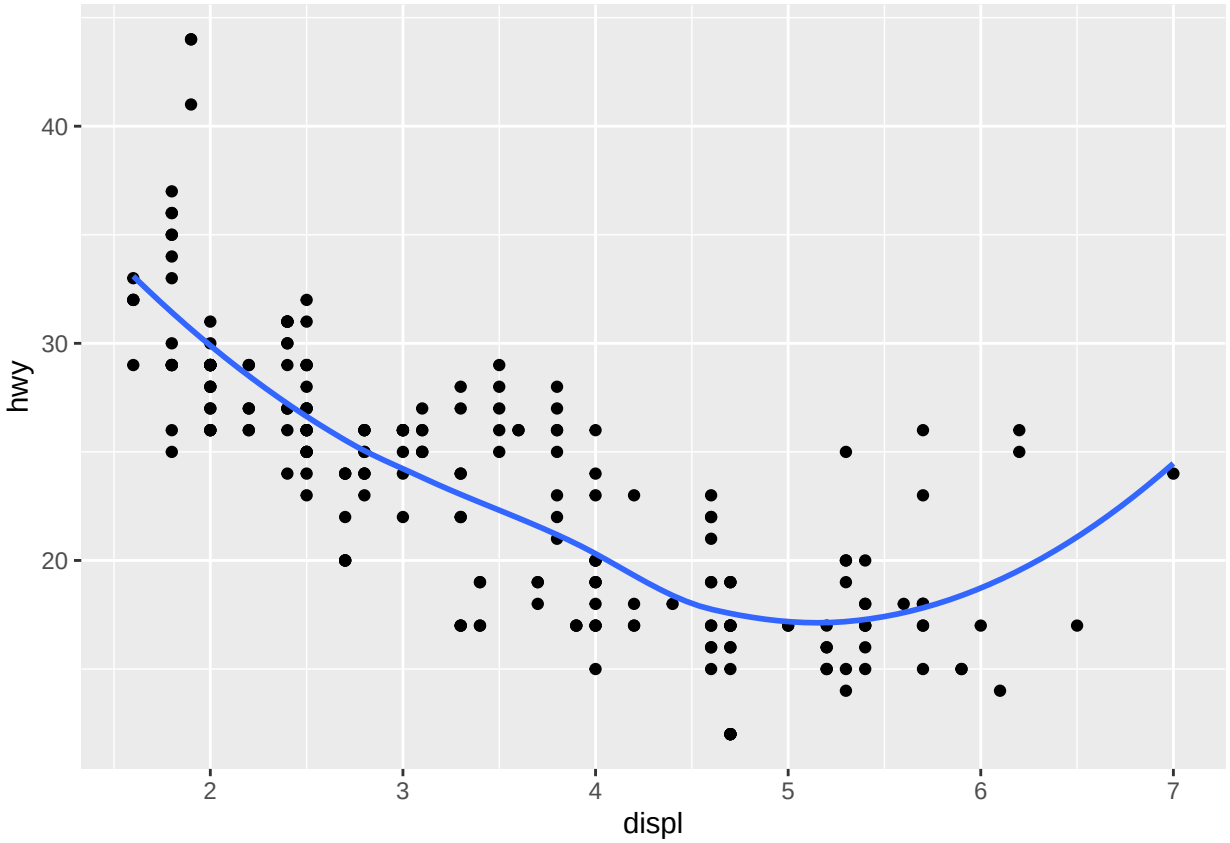
Section 9.3.1: Problem 4

Q) Recreate the R code necessary to generate the following graphs. Note that wherever a categorical variable is used in the plot, it's drv.

Ans: The code for each of the plots is given below.

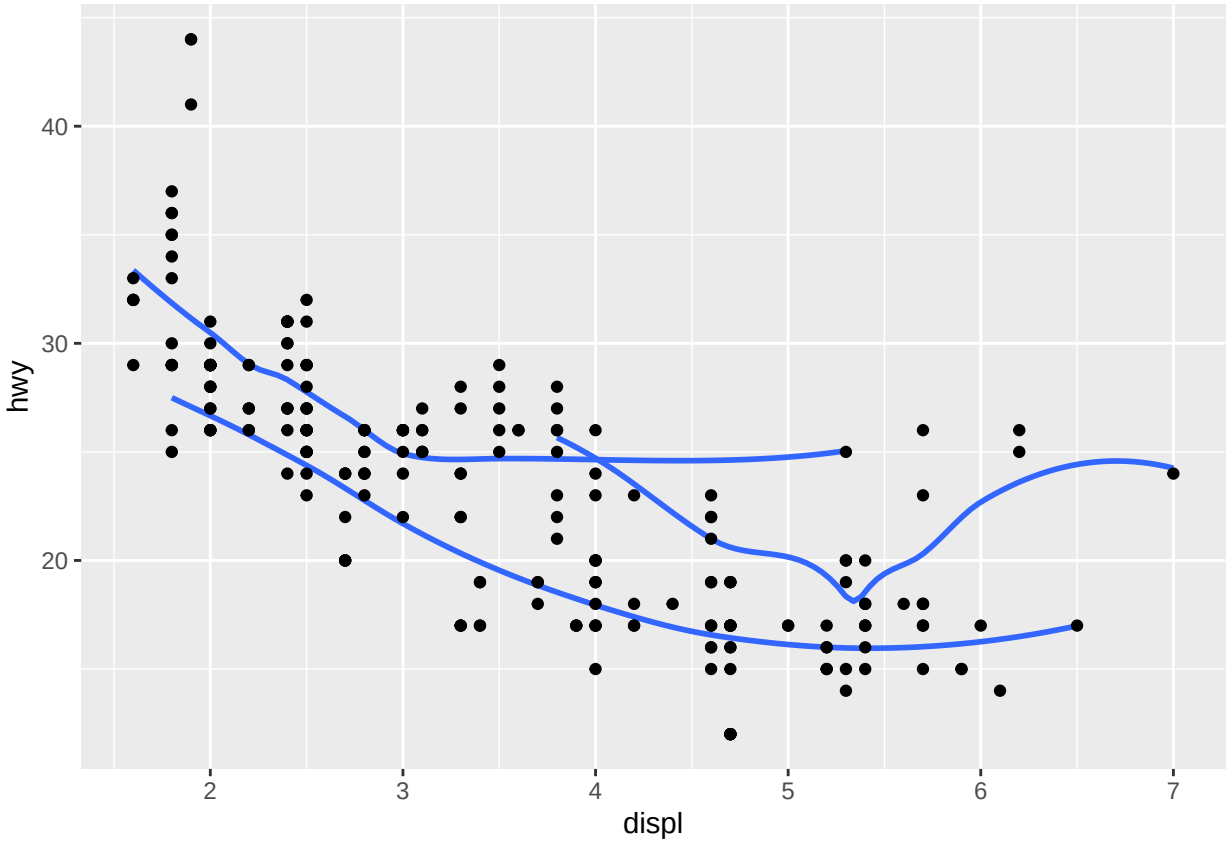
```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth(se = FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



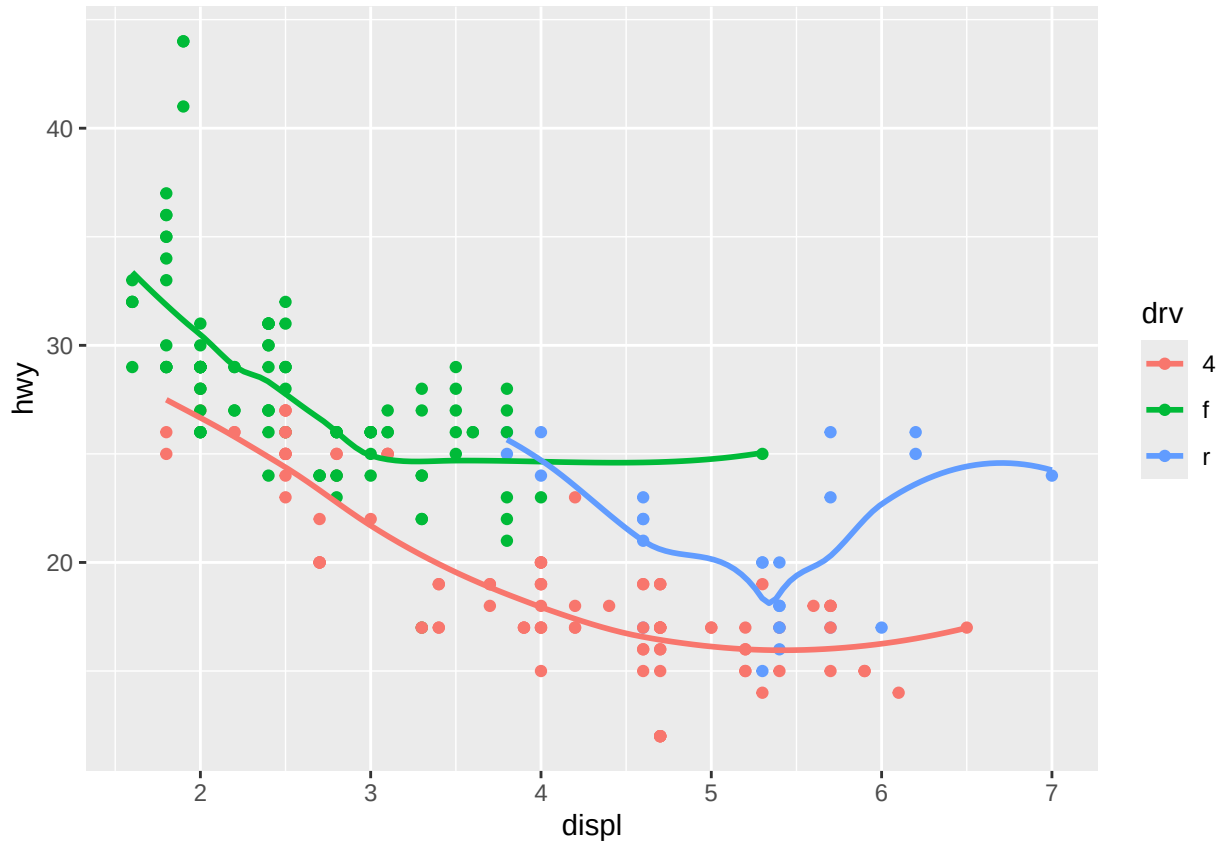

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(group = drv), se = FALSE) +
  geom_point()
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



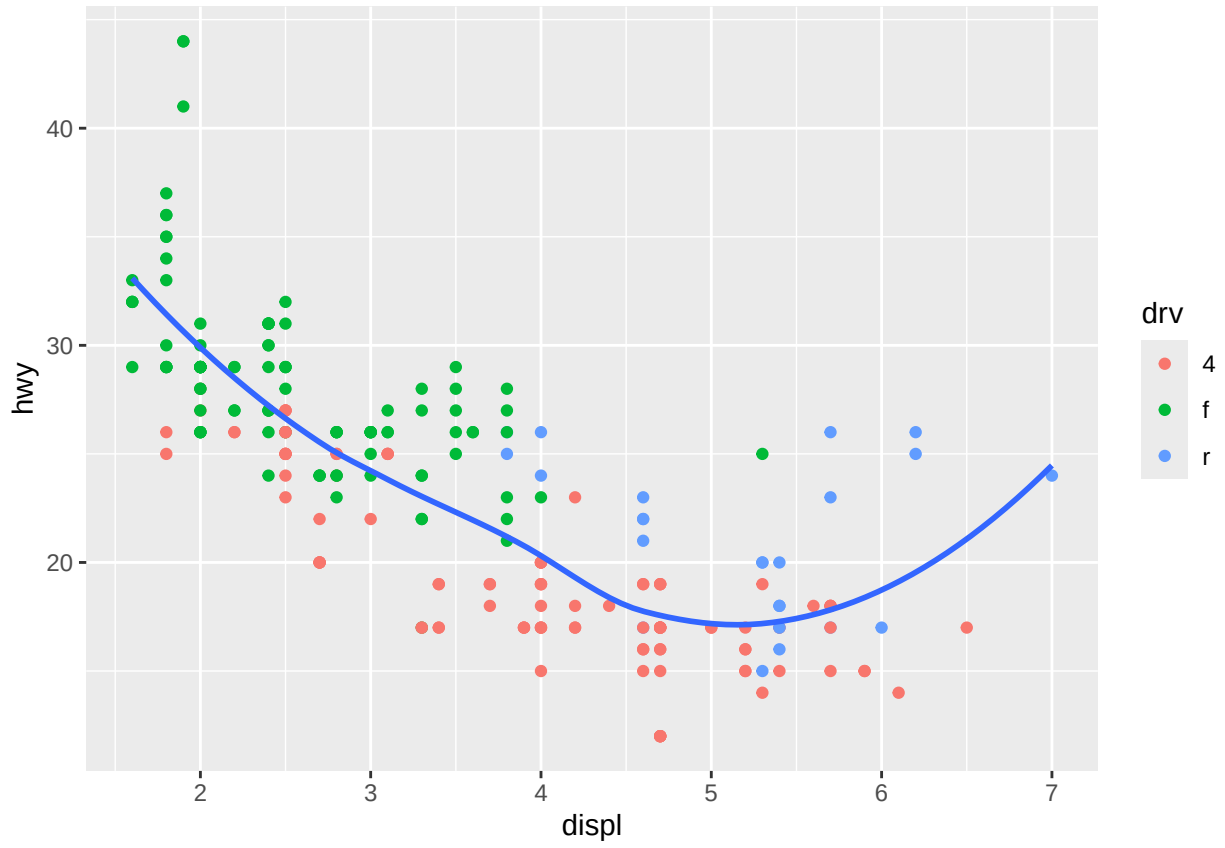

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  geom_smooth(se = FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



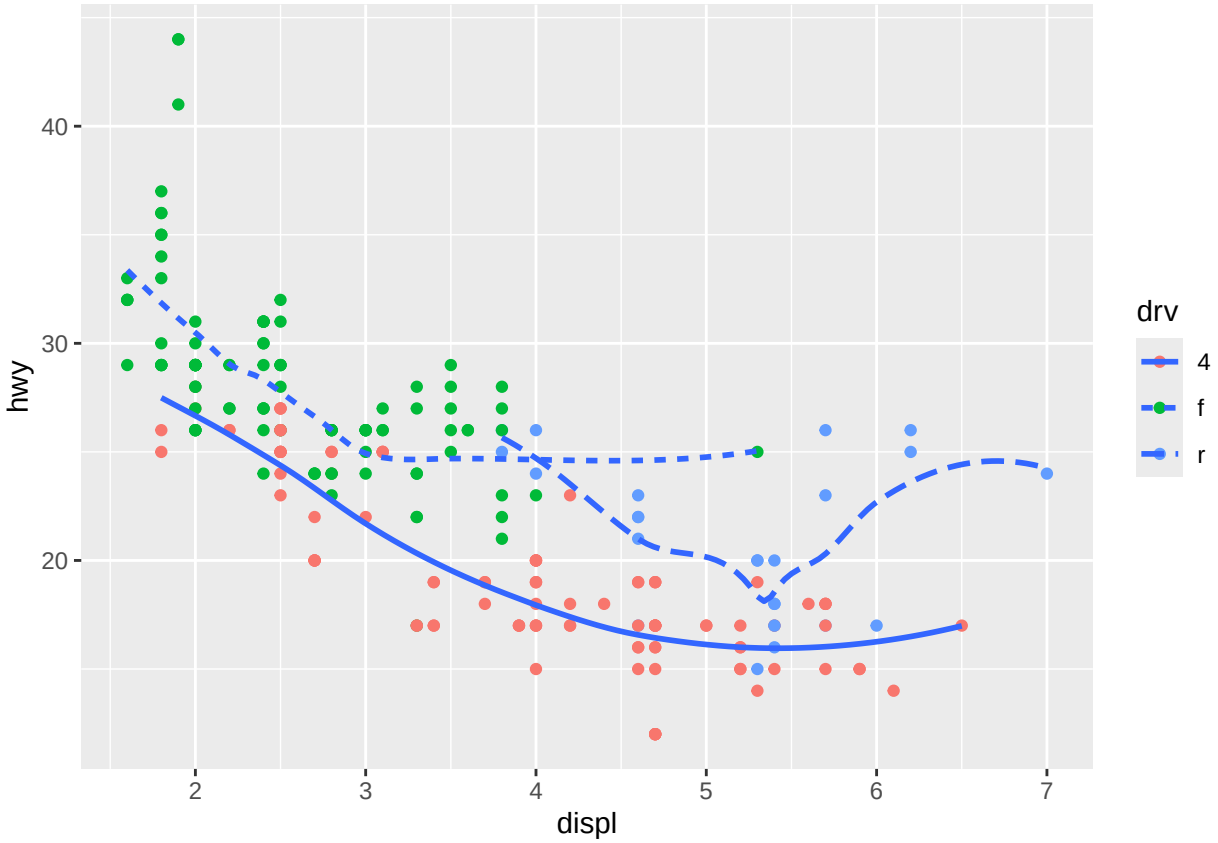

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(se = FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

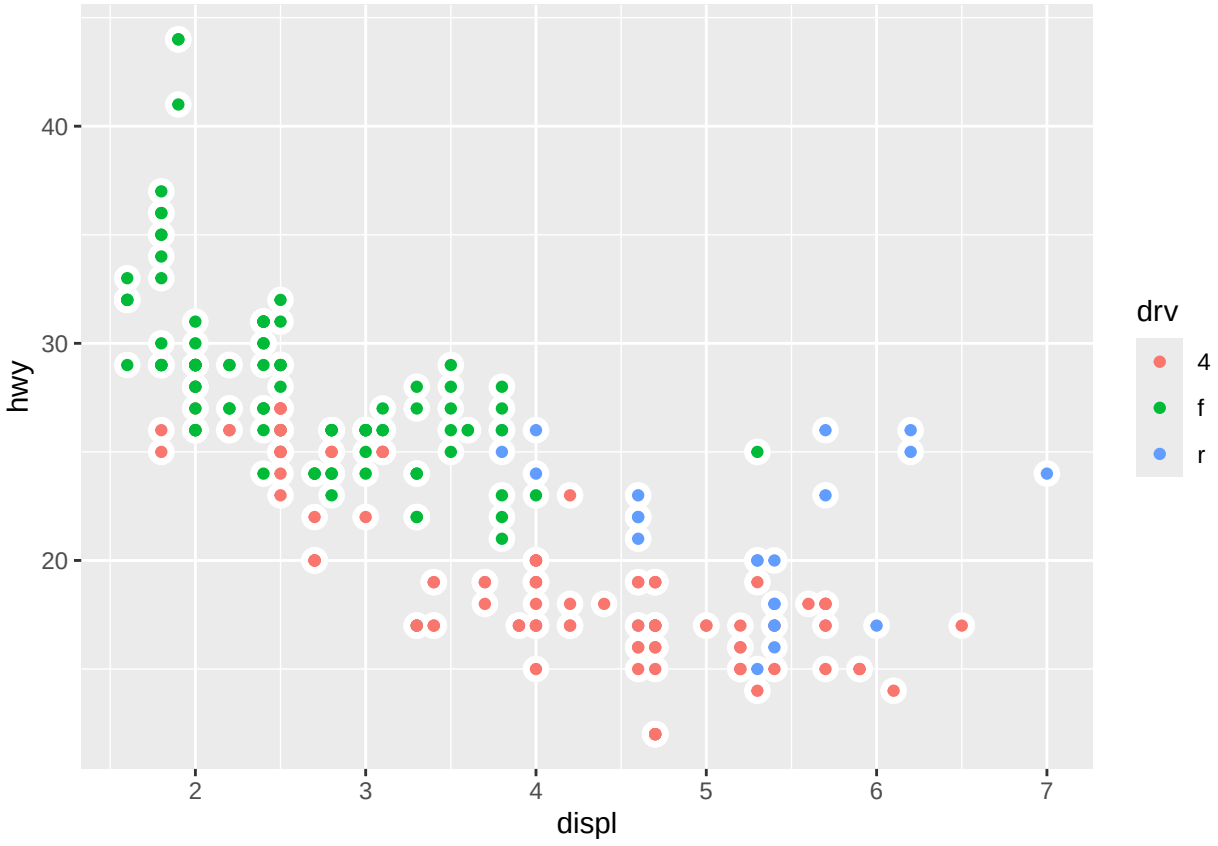



```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(aes(linetype = drv), se = FALSE)
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```




```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(size = 4, color = "white") +
  geom_point(aes(color = drv))
```

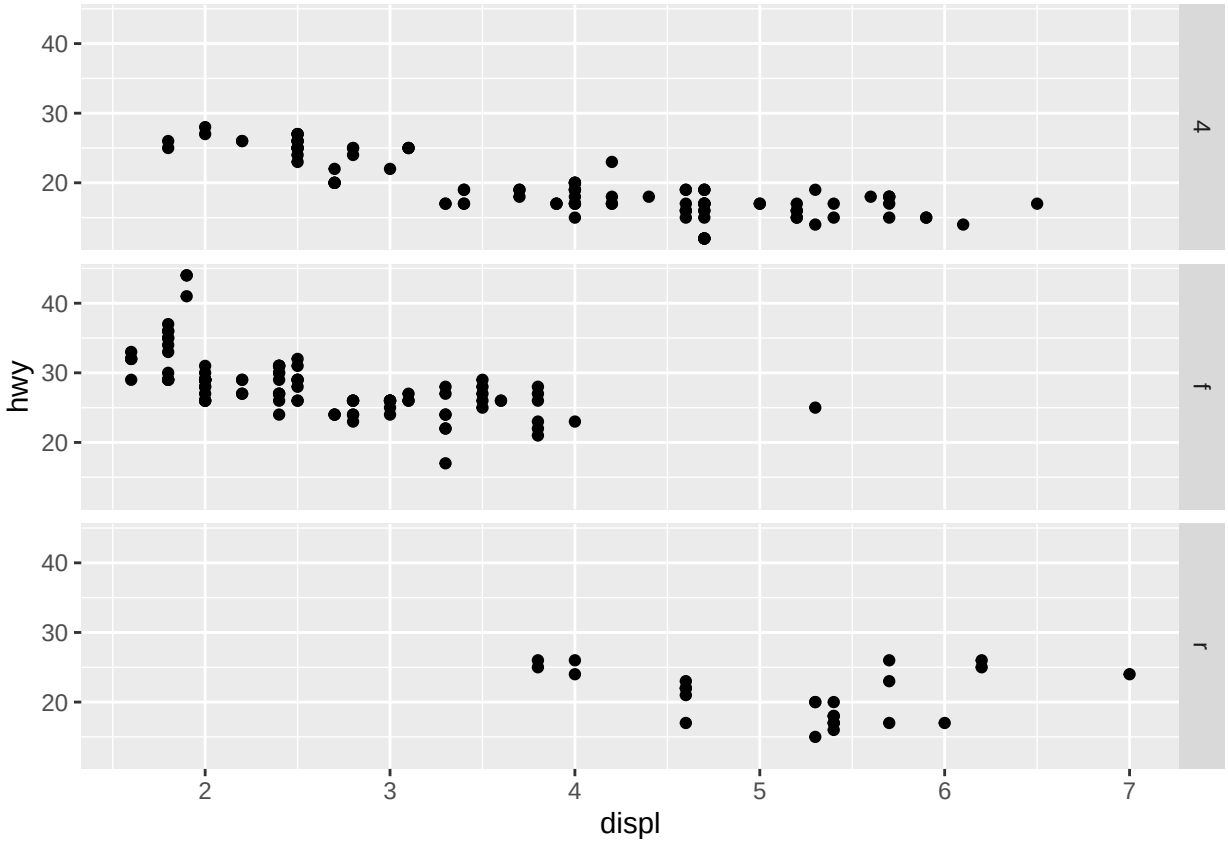


Section 9.4.1: Problem 3

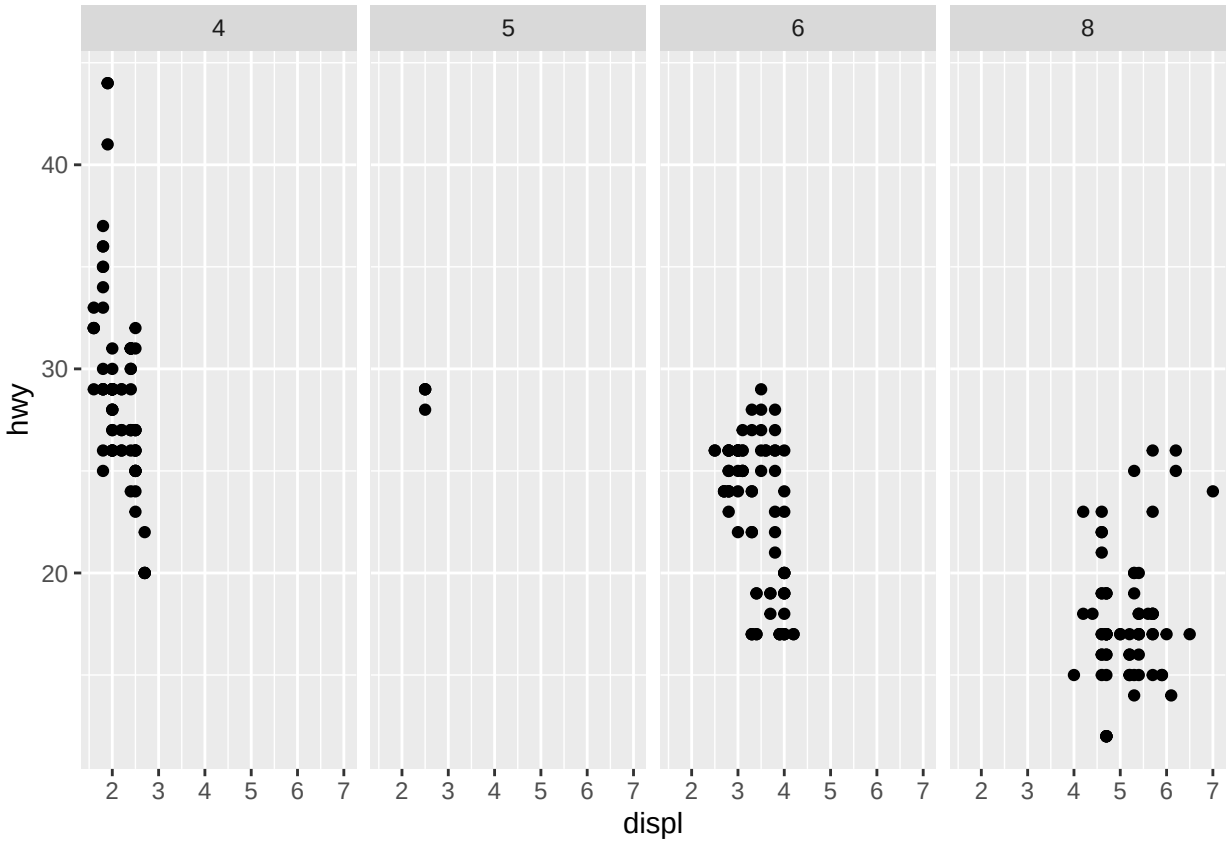
Q) What plots does the following code make? What does . do?

Given Code:

```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_grid(drv ~ .)
```




```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_grid(. ~ cyl)
```

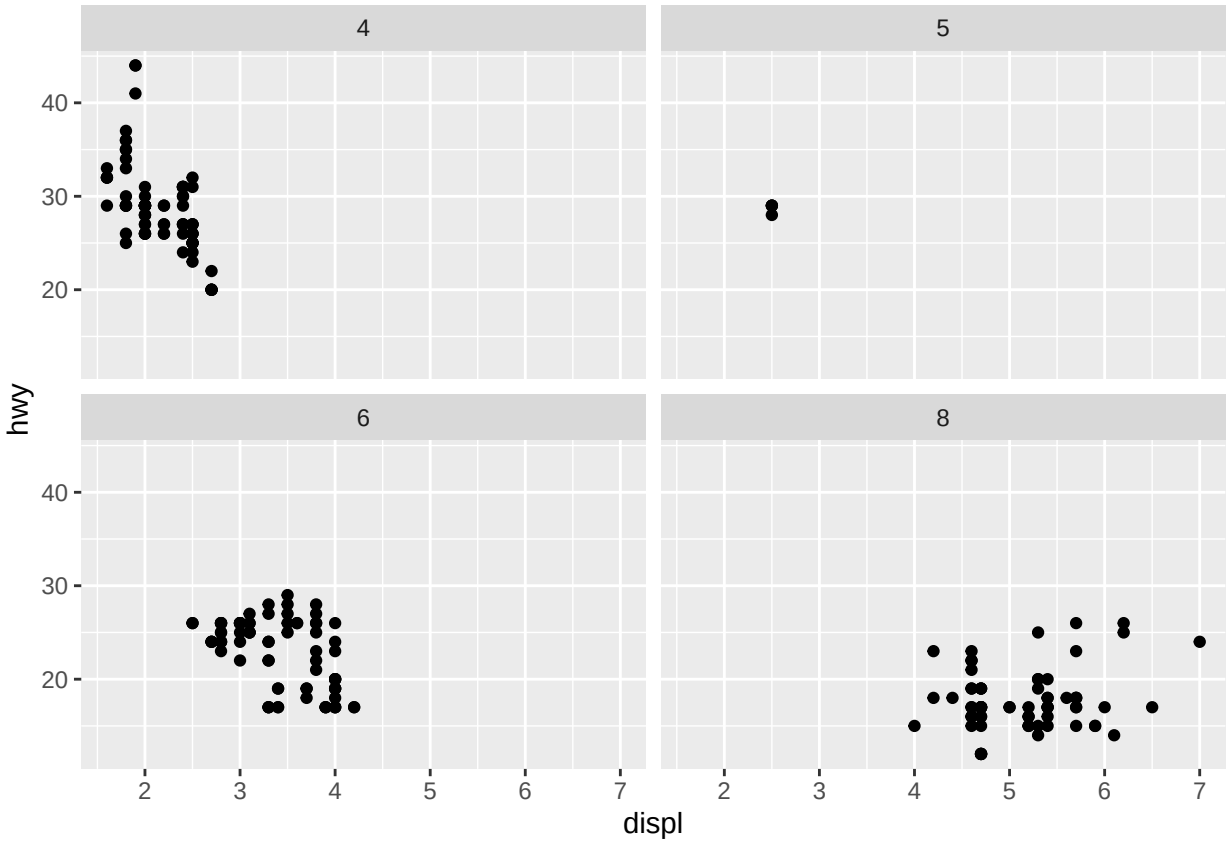


Ans: In the first plot, with facet_grid(drv ~ .), the period means "don't facet across columns". In the second plot, with facet_grid(. ~ drv), the period means "don't facet across rows". In general, the period means "keep everything together".

Section 9.4.1: Problem 4

Q) Take the first faceted plot in this section:

```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_wrap(~ cyl, nrow = 2)
```

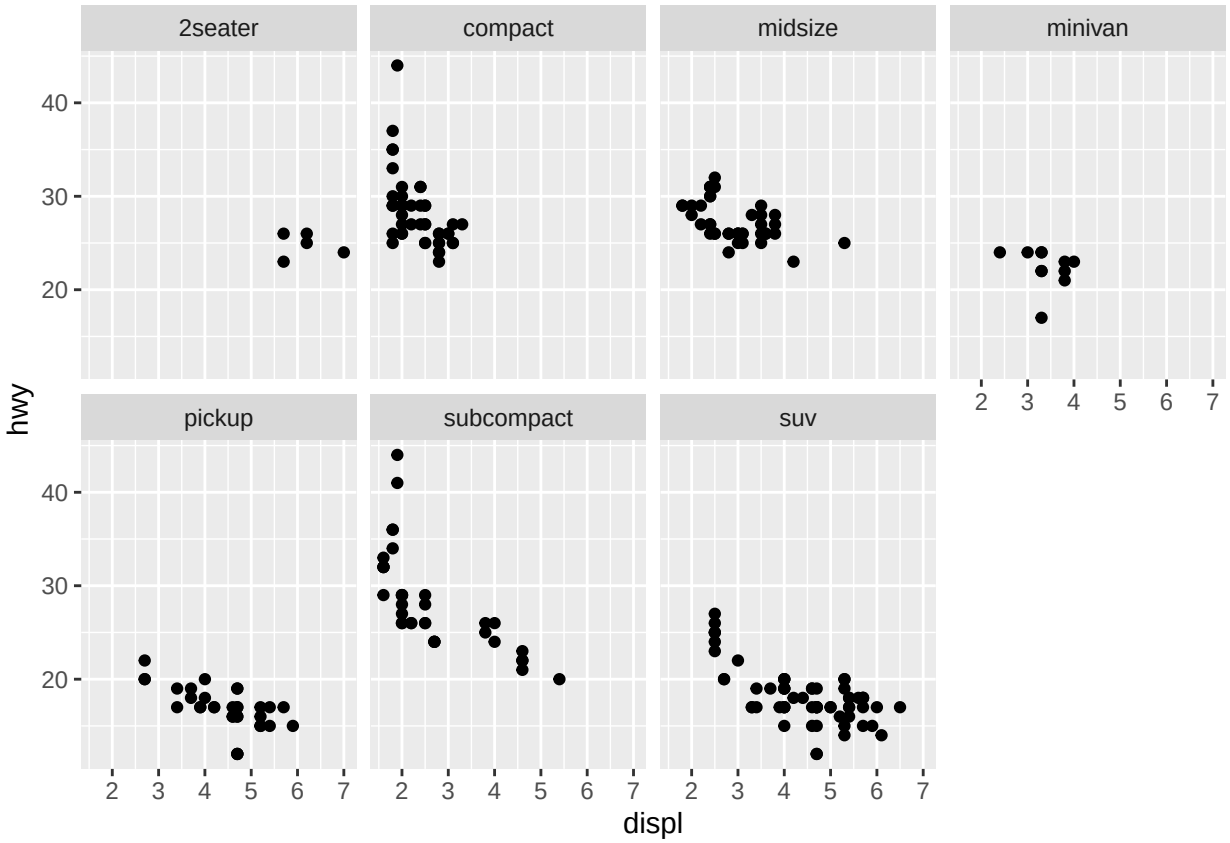


What are the advantages to using faceting instead of the color aesthetic? What are the disadvantages? How might the balance change if you had a larger dataset?

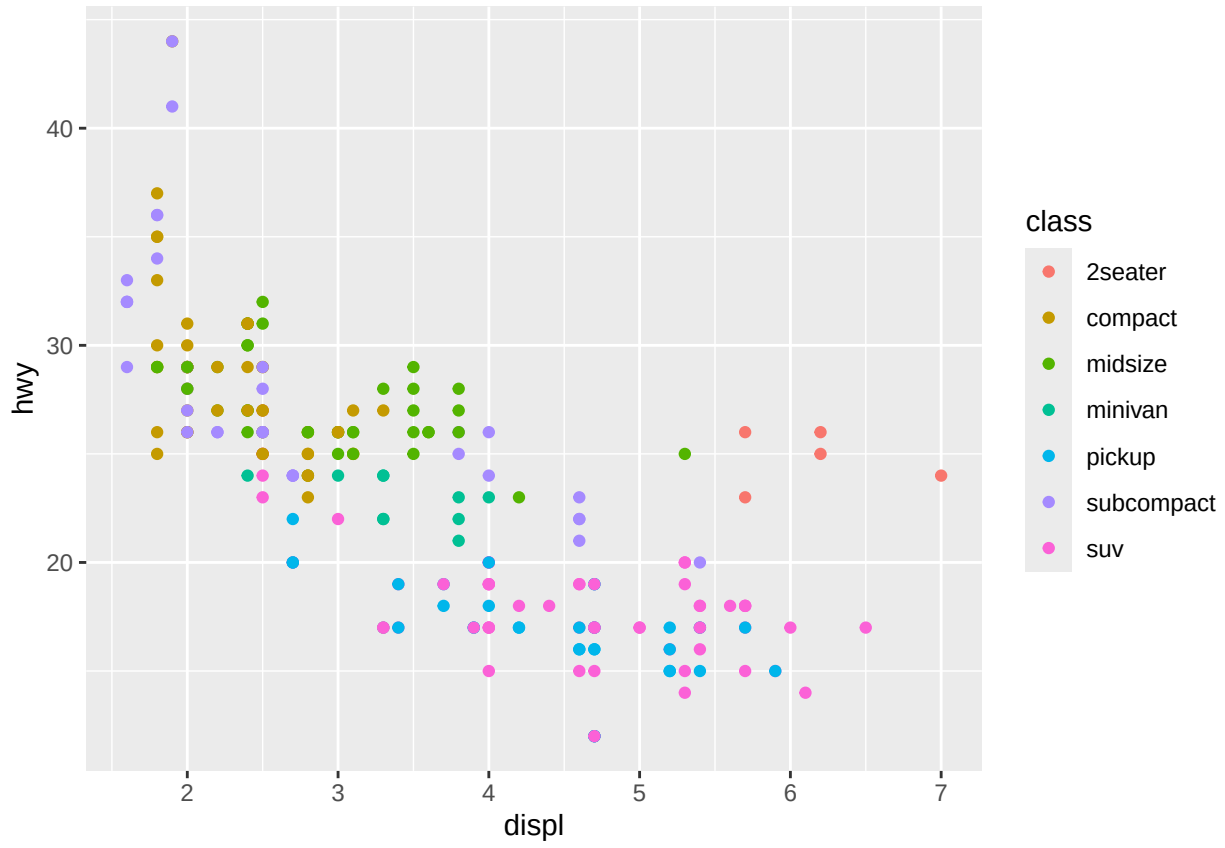
Ans: The advantages of faceting is seeing each class of car separately, without any overplotting. The disadvantage is not being able to compare the classes to each other as easily when they're in separate plots. Additionally, color can be helpful for easily telling classes apart. Using both can be helpful, but doesn't mitigate the issue of easy comparison across classes. If we were interested in a specific class, e.g. compact cars, it would be useful to highlight that group only with an additional layer as shown in the last plot below.

Example:

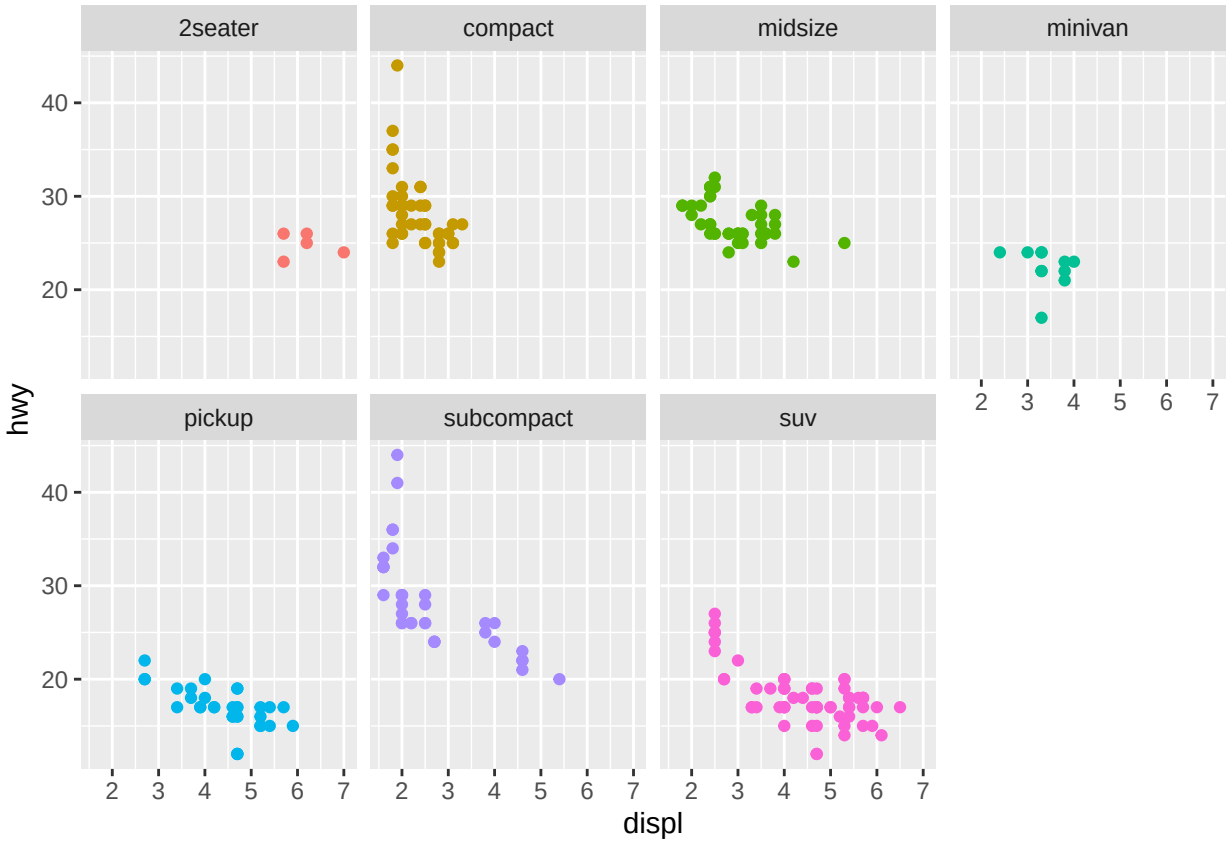
```
# facet
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy)) +
  facet_wrap(~ class, nrow = 2)
```



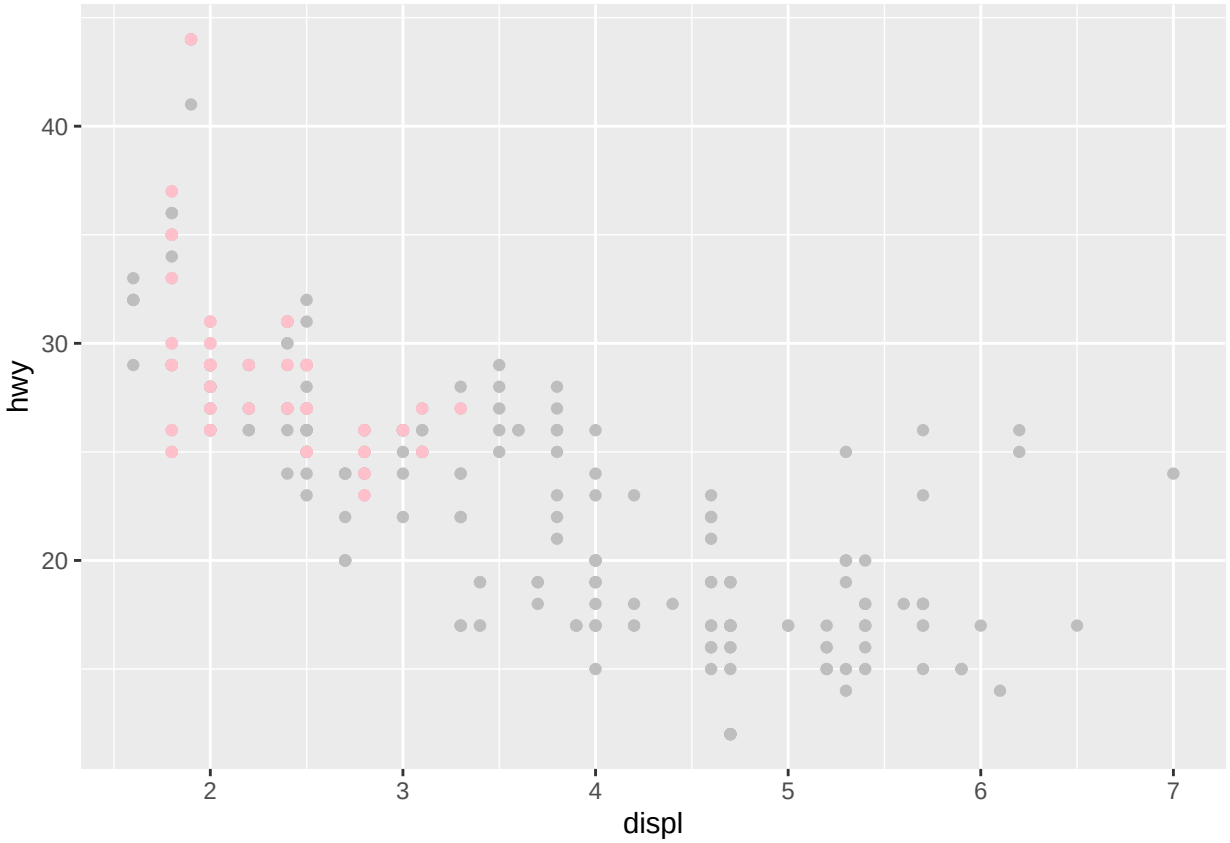

```
# color
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy, color = class))
```




```
# both
ggplot(mpg) +
  geom_point(
    aes(x = displ, y = hwy, color = class),
    show.legend = FALSE) +
  facet_wrap(~ class, nrow = 2)
```




```
# highlighting
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "gray") +
  geom_point(
    data = mpg |> filter(class == "compact"),
    color = "pink"
  )
```



Section 9.4.1: Problem 5

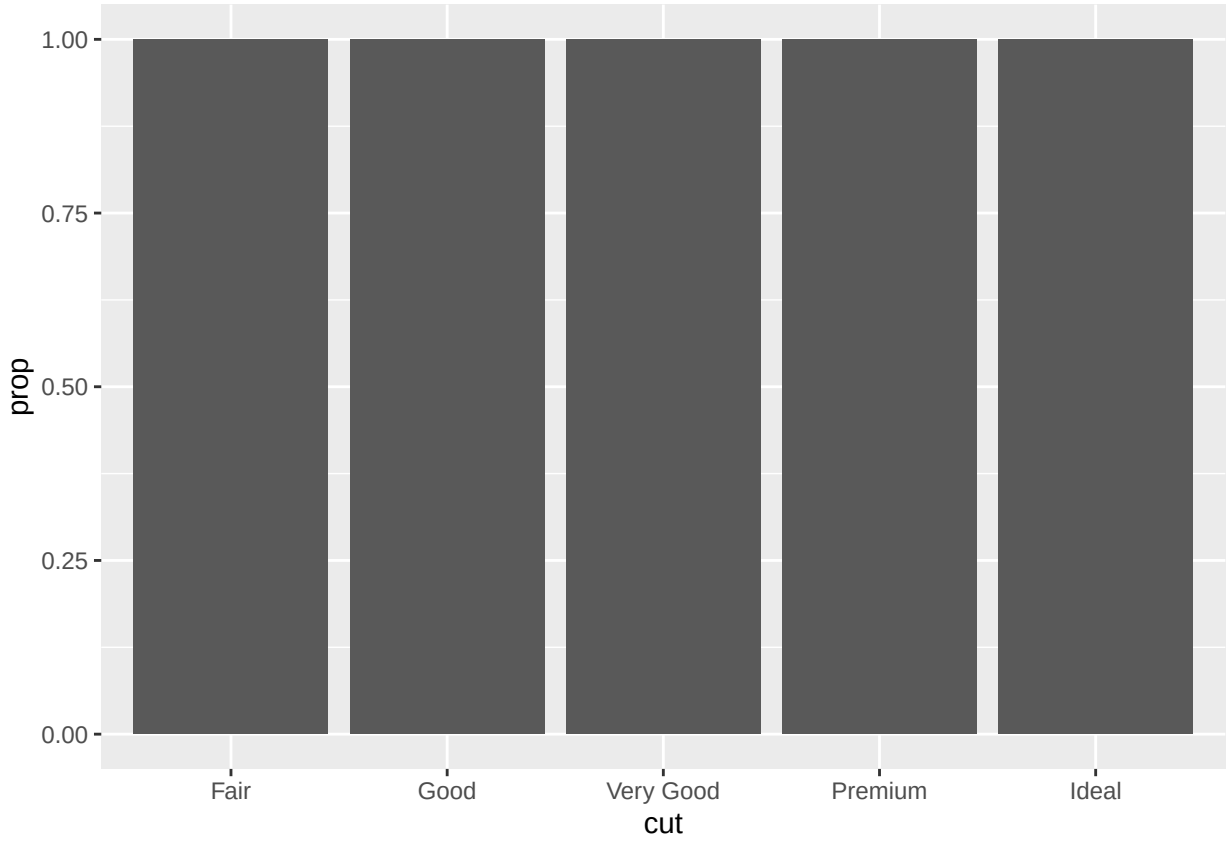
Q) Read ?facet_wrap. What does nrow do? What does ncol do? What other options control the layout of the individual panels? Why doesn't facet_grid() have nrow and ncol arguments?

Ans: nrow controls the number panels and ncol controls the number of columns the panels should be arranged in. facet_grid() does not have these arguments because the number of rows and columns are determined by the number of levels of the two categorical variables facet_grid() plots. dir controls the whether the panels should be arranged horizontally or vertically.

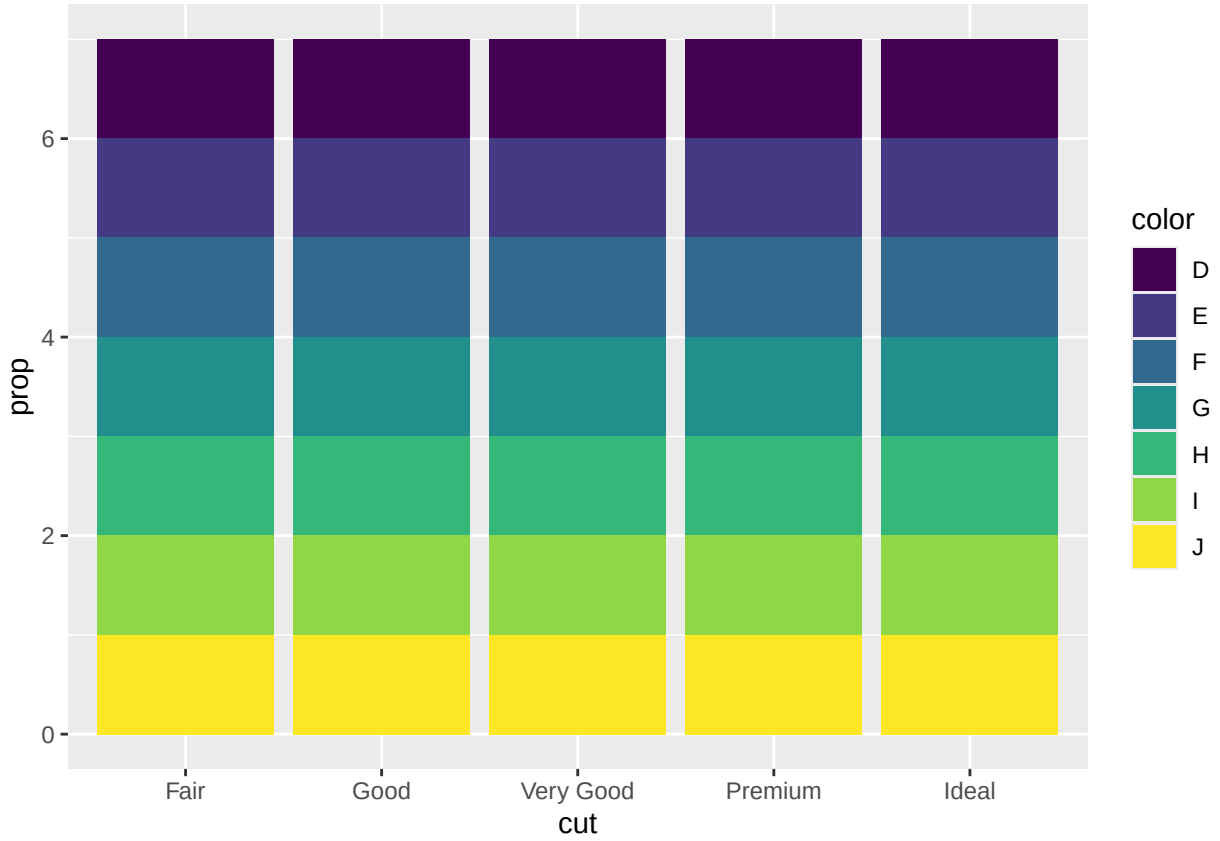
Section 9.5.1: Problem 5

Q) In our proportion bar chart, we needed to set group = 1. Why? In other words, what is the problem with these two graphs?

```
ggplot(diamonds, aes(x = cut, y = after_stat(prop))) +
  geom_bar()
```

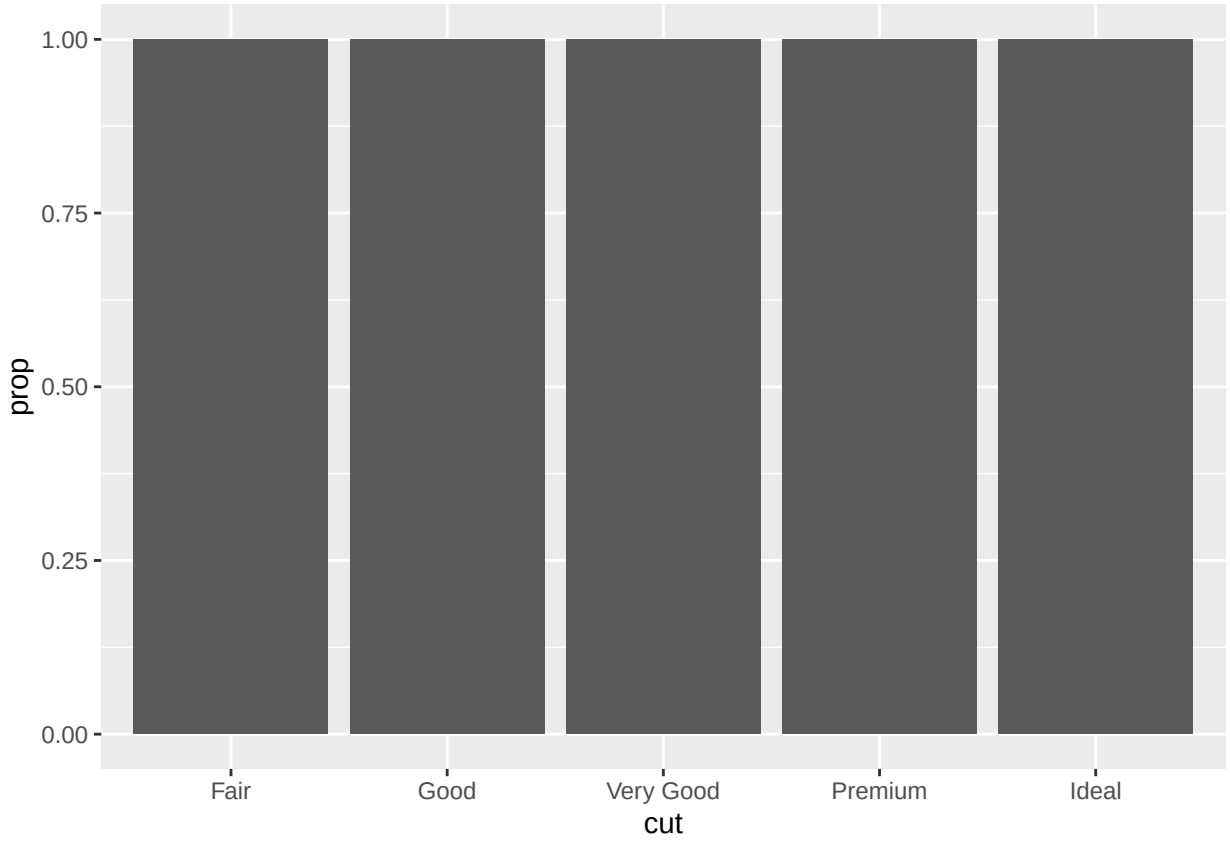



```
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop))) +
  geom_bar()
```

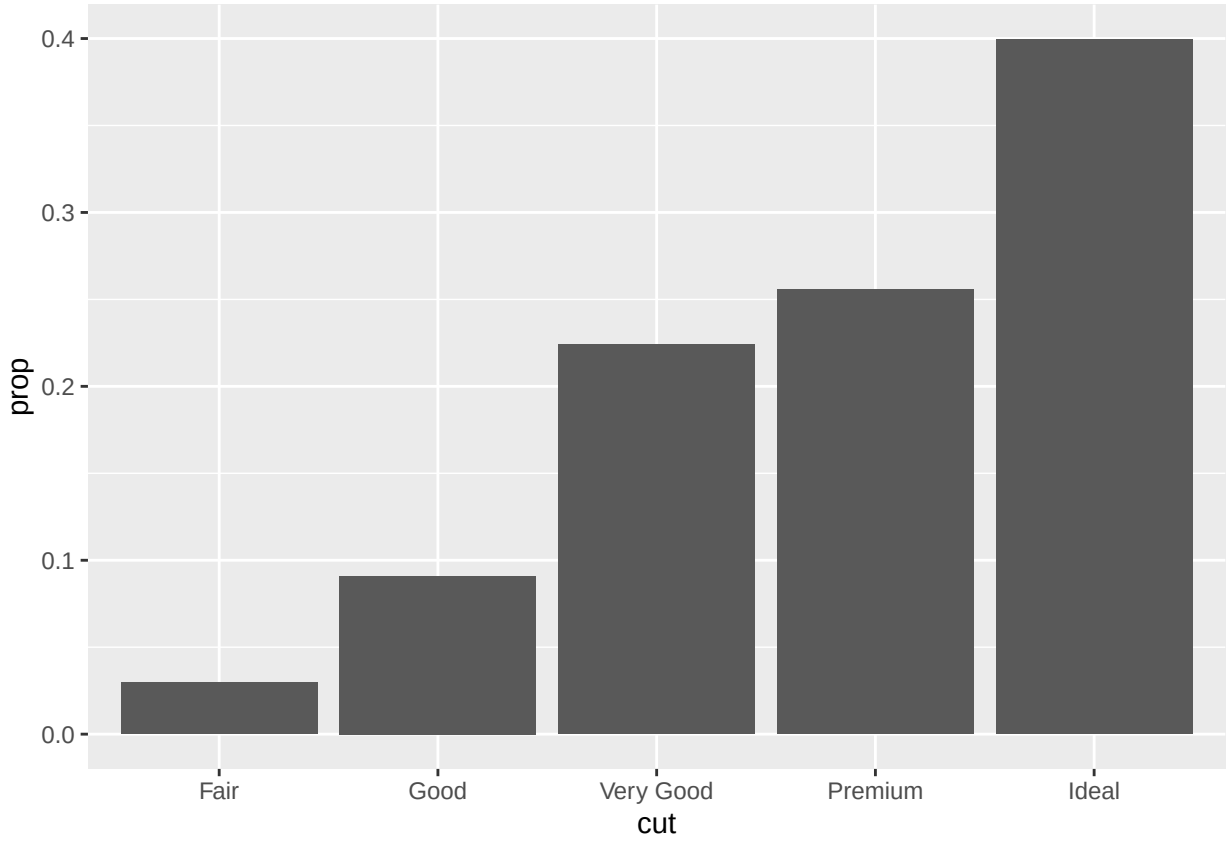


Ans: In the first pair of plots, we see that setting group = 1 results in the marginal proportions of cuts being plotted. In the second pair of plots, setting group = color results in the proportions of colors within each cut being plotted.

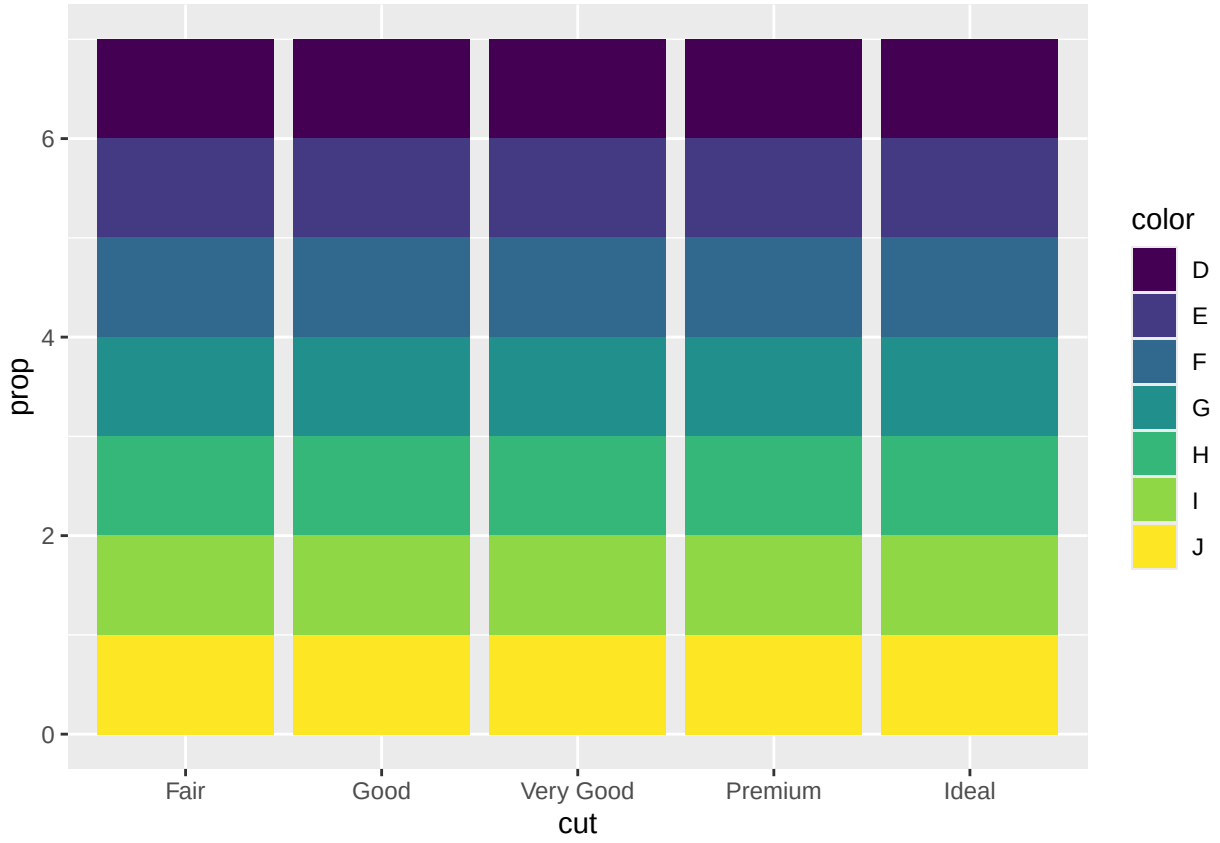
```
# one variable
ggplot(diamonds, aes(x = cut, y = after_stat(prop))) +
  geom_bar()
```



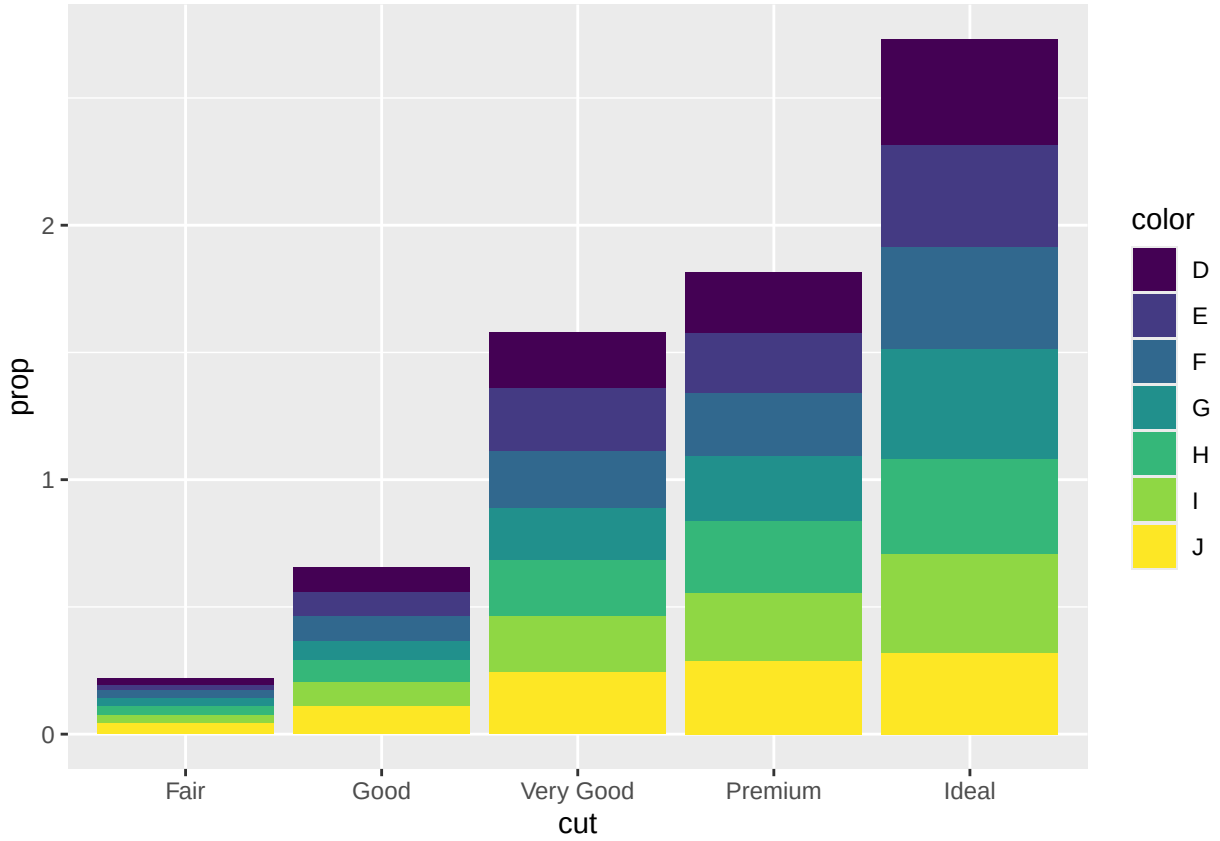

```
ggplot(diamonds, aes(x = cut, y = after_stat(prop), group = 1)) +
  geom_bar()
```




```
# two variables
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop))) +
  geom_bar()
```




```
ggplot(diamonds, aes(x = cut, fill = color, y = after_stat(prop), group = color)) +
  geom_bar()
```

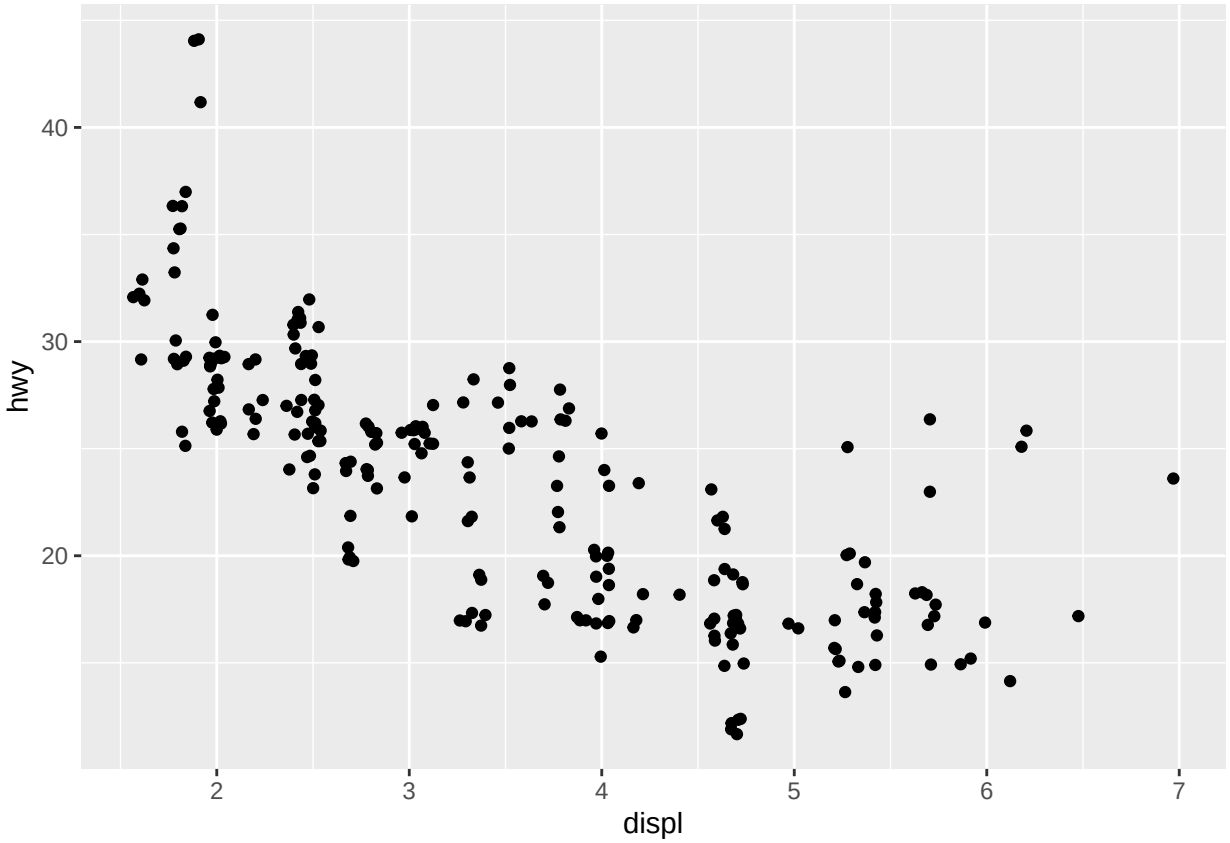


Section 9.6.1: Problem 4

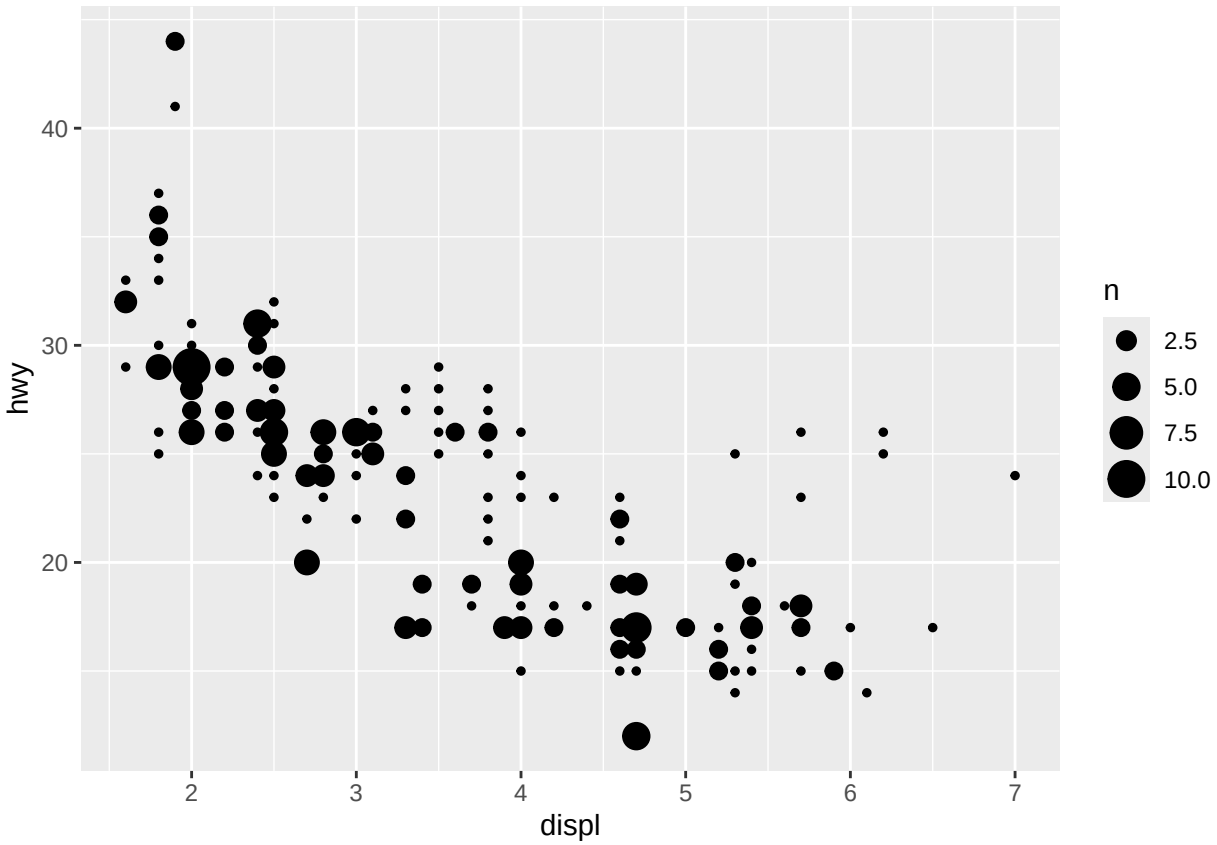
Q) Compare and contrast geom_jitter() with geom_count().

Ans: geom_jitter() adds random noise to the location of the points to avoid overplotting. geom_count() sizes the points based on the number of observations at a given location.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_jitter()
```




```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_count()
```



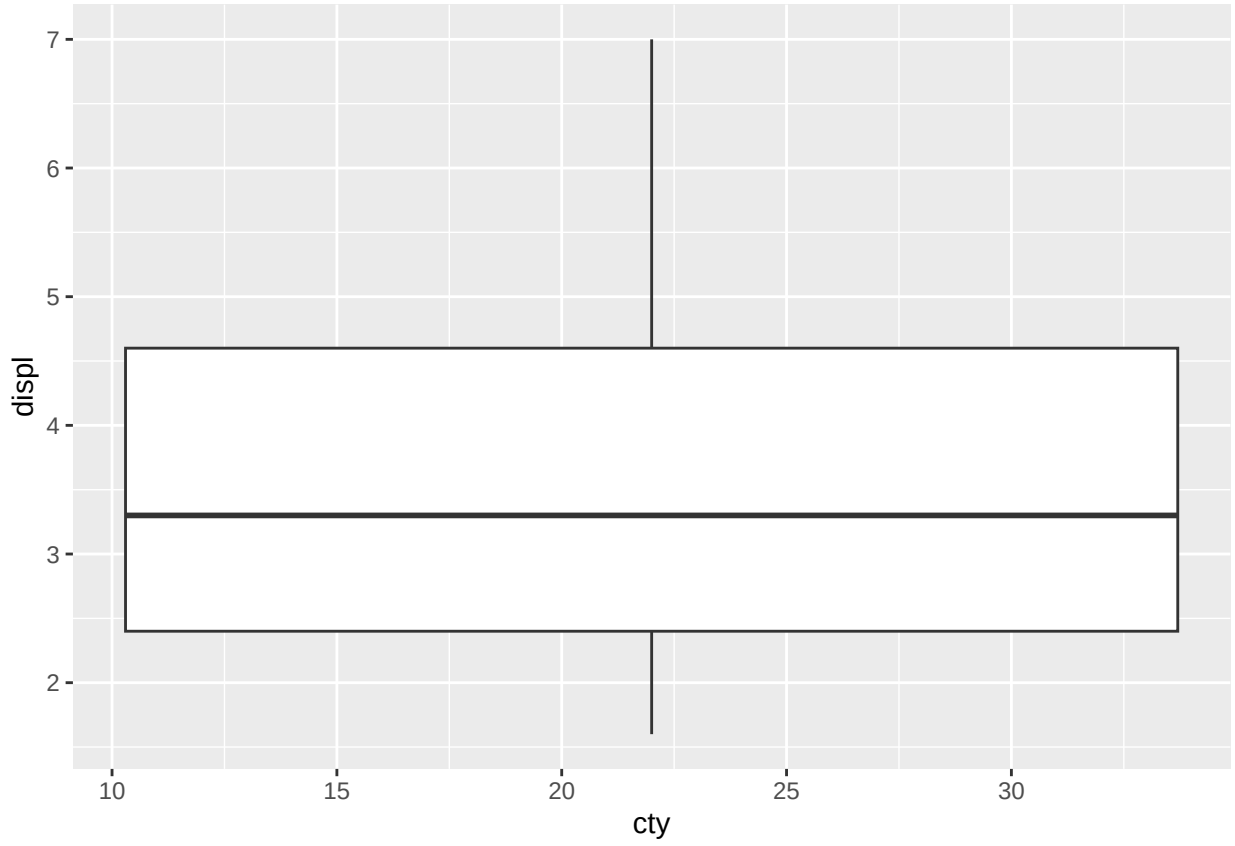
Section 9.6.1: Problem 5

Q) What's the default position adjustment for geom_boxplot()? Create a visualization of the mpg dataset that demonstrates it.

Ans: The default is position for geom_boxplot() is "dodge2".

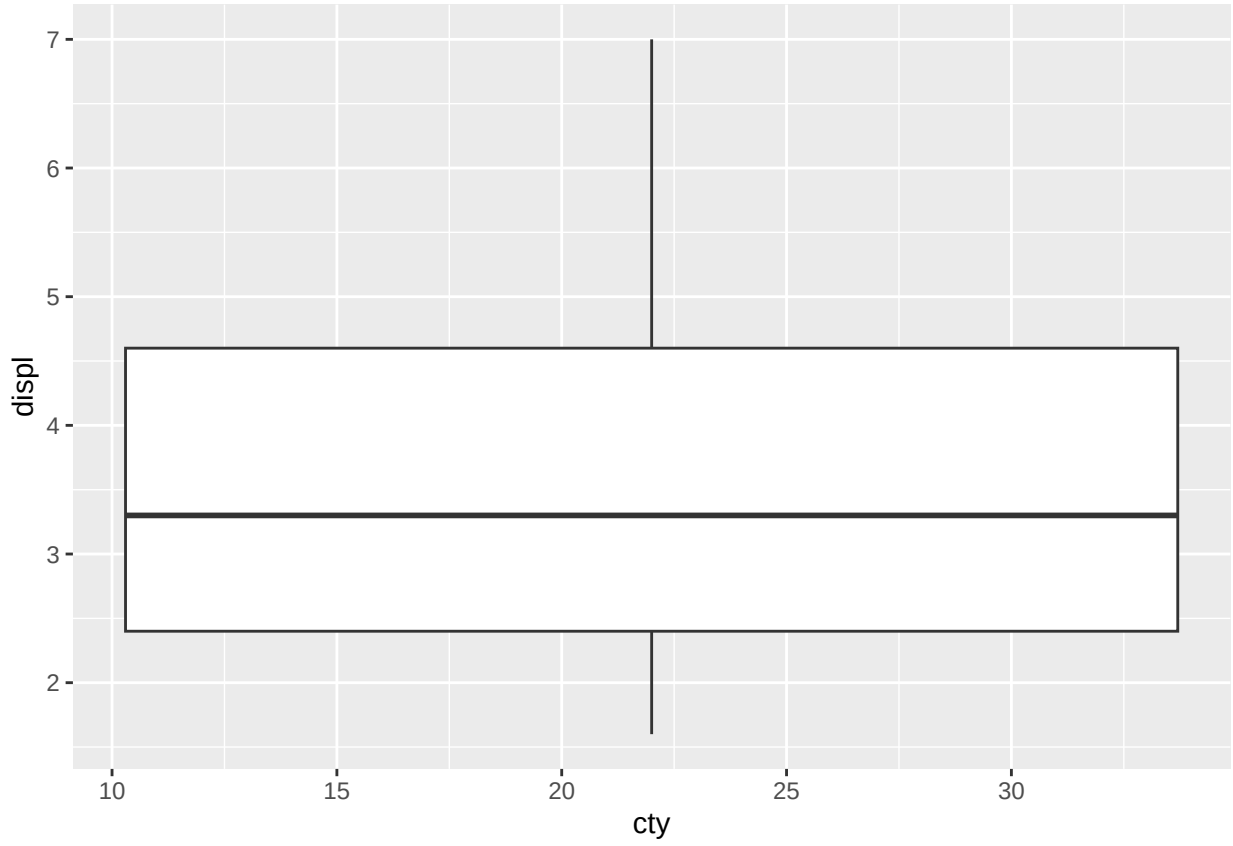
```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_boxplot()
```

```
## Warning: Continuous x aesthetic
## i did you forget `aes(group = ...)`?
```




```
ggplot(mpg, aes(x = cty, y = displ)) +
  geom_boxplot(position = "dodge2")
```

```
## Warning: Continuous x aesthetic
## i did you forget `aes(group = ...)`?
```



Section 10.3.3: Problem 3

Q) How many diamonds are 0.99 carat? How many are 1 carat? What do you think is the cause of the difference?

Ans:

There are 23 diamonds weighing 0.99 carats.

There are 1,558 diamonds weighing exactly 1.00 carat.

- Cause: Diamonds are often cut and marketed to hit "milestone weights" — round numbers like 1.00 carat are considered more desirable and valuable.
- Jewelers and cutters may intentionally cut slightly larger diamonds down to exactly 1.00 carat, even if that means losing a bit of weight, because 1.00 ct sounds much more appealing to buyers than 0.99 ct.
- Thus, there's an artificial spike at 1.00 carat and a drop just below it (0.99 ct).

```
library(tidyverse)

diamonds |>
  count(carat) |>
  filter(carat == 0.99 | carat == 1)
```

```
## # A tibble: 2 x 2
##   carat     n
```

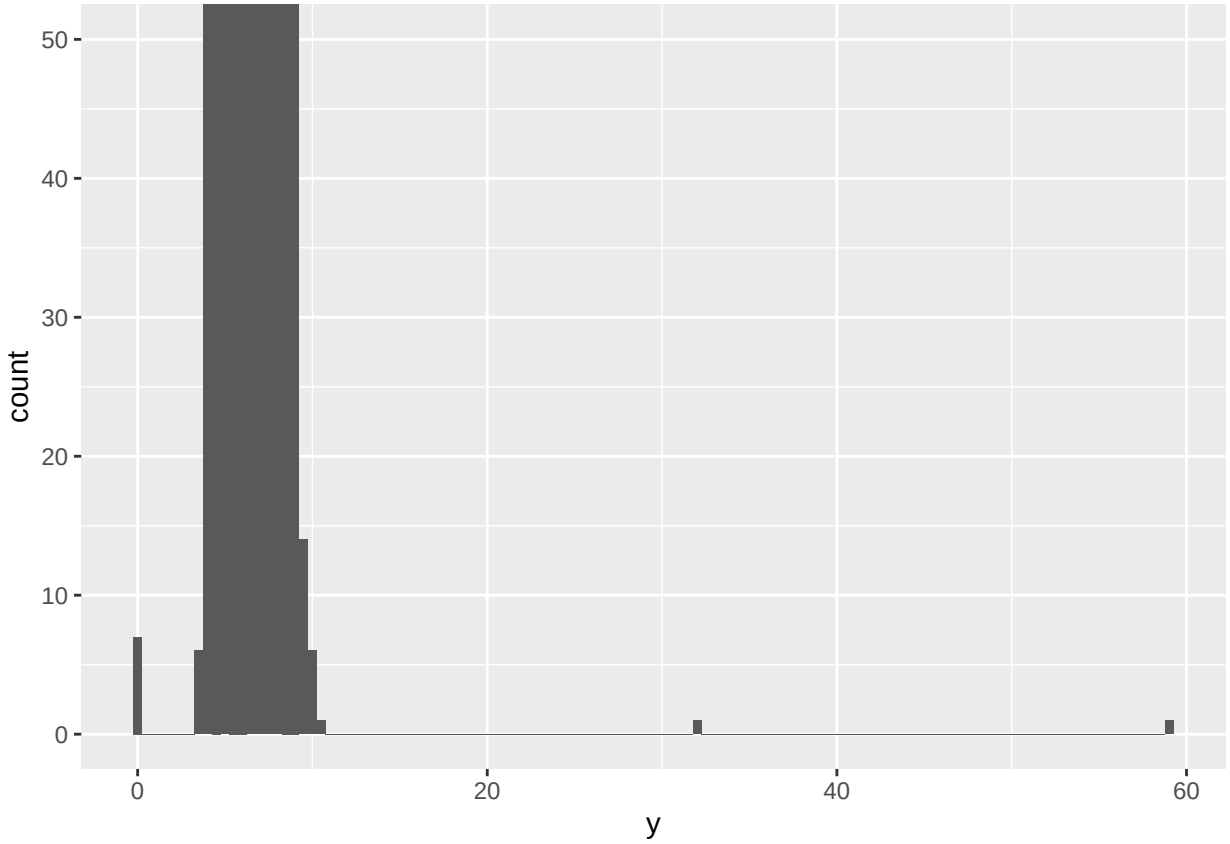
```
##   <dbl> <int>
## 1  0.99    23
## 2  1     1558
```

Section 10.3.3: Problem 4

Q) Compare and contrast coord_cartesian() vs. xlim() or ylim() when zooming in on a histogram. What happens if you leave binwidth unset? What happens if you try and zoom so only half a bar shows?

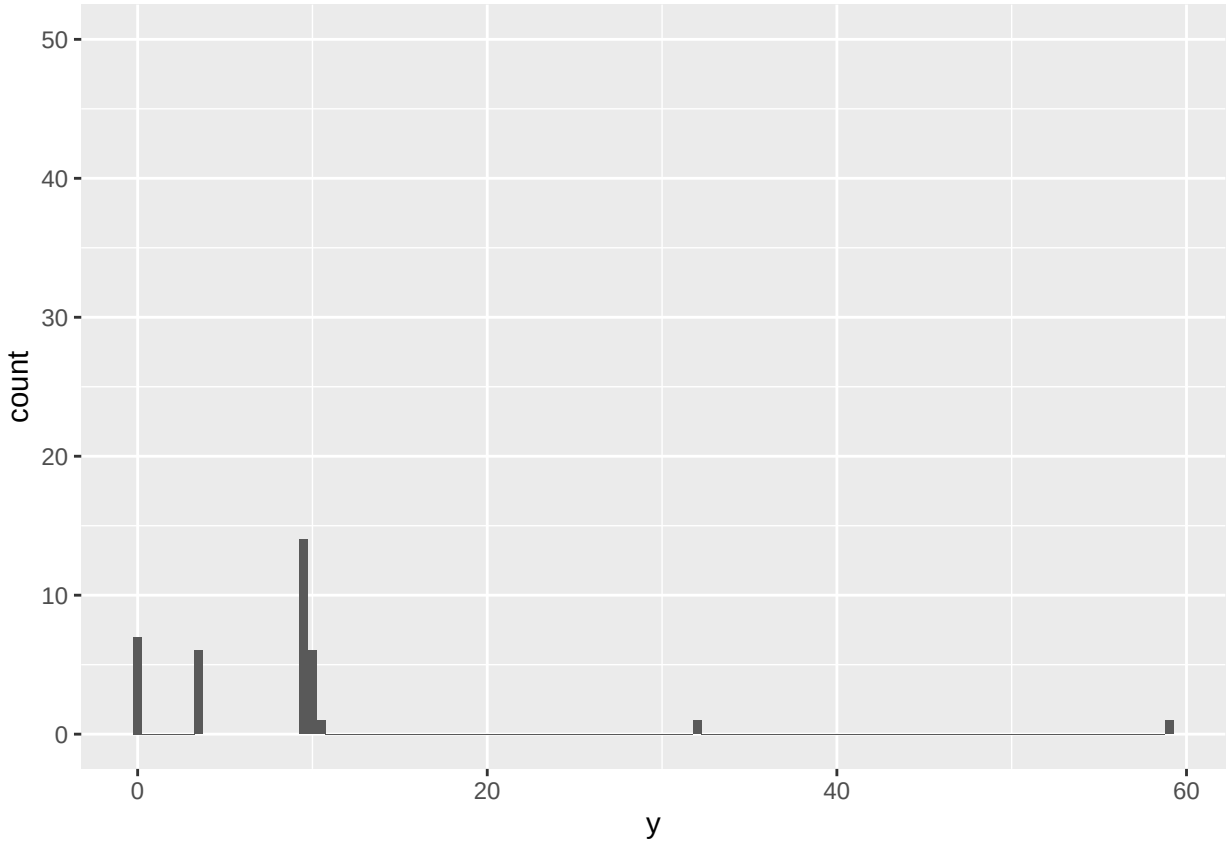
Ans:

```
# Zoom without losing data
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5) +
  coord_cartesian(ylim = c(0, 50))
```




```
# Filters out data outside 0-50 before binning
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5) +
  ylim(0, 50)
```

```
## Warning: Removed 11 rows containing missing values or values outside the scale range
## (`geom_bar()`).
```



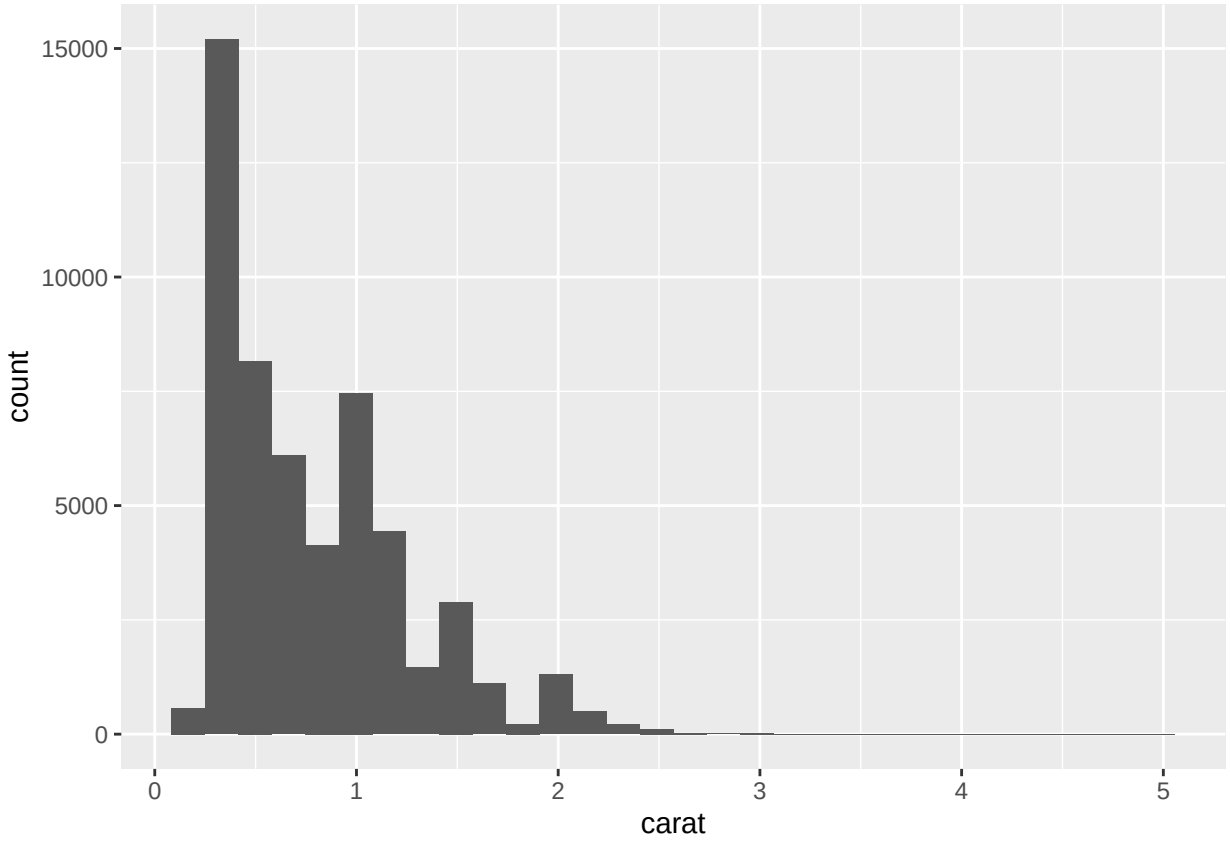
- With coord_cartesian(), the bins still reflect all data — just cropped visually.
- With ylim(), out-of-range values are excluded before binning, so bar heights and proportions can change.

What if you leave binwidth unset?

- ggplot2 automatically chooses a binwidth based on the data's range and size (using the Freedman–Diaconis rule).
- The default choice may be fine for large datasets, but it can sometimes hide fine details or over-smooth the distribution.
- You might see warning messages like: "stat_bin() using bins = 30. Pick better value with binwidth."

```
ggplot(diamonds, aes(x = carat)) + geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



What happens if you zoom so only half a bar shows?

- If you use coord_cartesian() to zoom in so that only part of a bar (bin) is visible:
  - The bar still exists fully, but only the visible part is shown.
  - It looks like a "cut" bar, but ggplot2 hasn't recalculated the bin — it's just a viewport crop.
- If you use xlim()/ylim() instead:
  - That half bar will disappear entirely if the bin's midpoint falls outside the filtered range, because the data used to build that bar was removed.

Section 10.5.1.1: Problem 5

Q) Create a visualization of diamond prices vs. a categorical variable from the diamonds dataset using geom_violin(), then a faceted geom_histogram(), then a colored geom_freqpoly(), and then a colored geom_density(). Compare and contrast the four plots. What are the pros and cons of each method of visualizing the distribution of a numerical variable based on the levels of a categorical variable?

Ans:

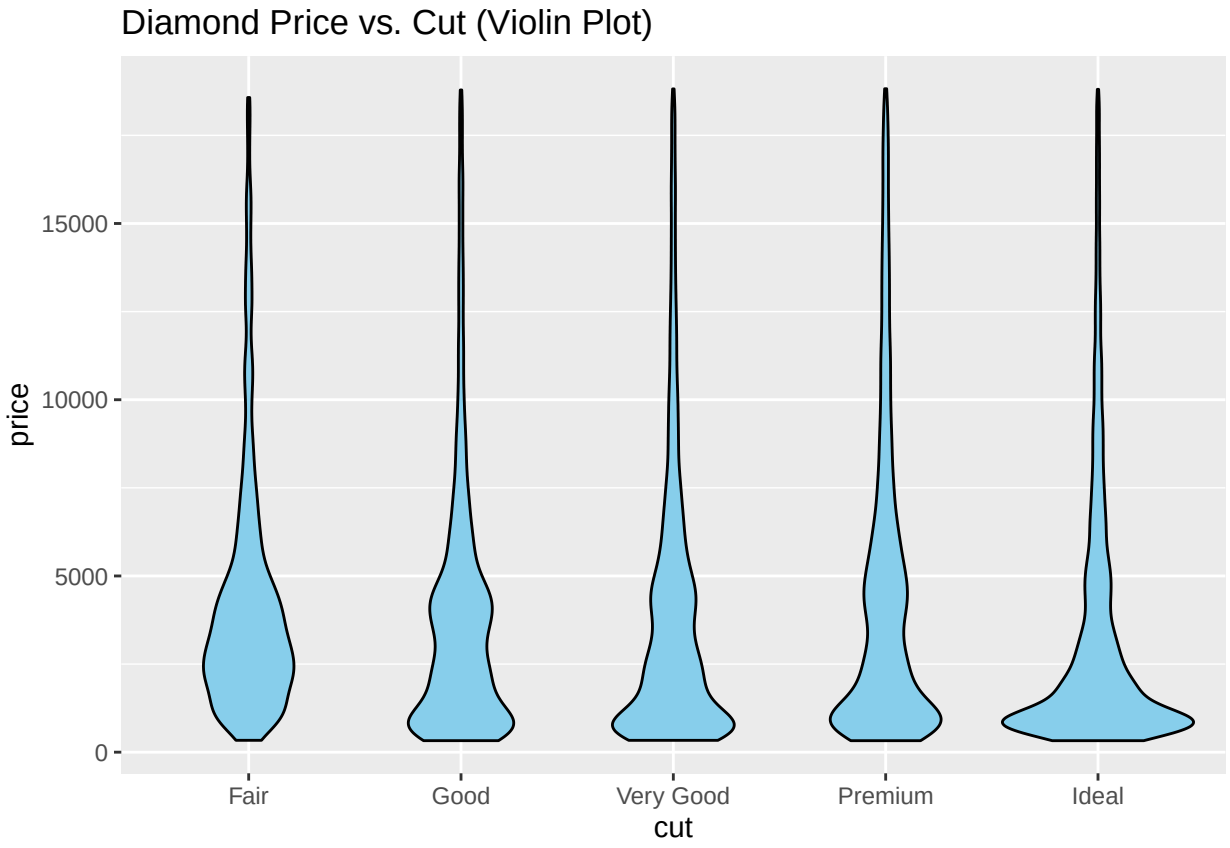
(a) Violin plot

- **Pros:**
  - Shows both the distribution shape and spread for each category.

  - Combines boxplot and density information in a compact form.

- **Cons:**
  - Harder to interpret exact frequencies or medians.
  - Overlapping shapes can obscure fine details or outliers.

```
ggplot(diamonds, aes(x = cut, y = price)) +
  geom_violin(fill = "skyblue", color = "black") +
  labs(title = "Diamond Price vs. Cut (Violin Plot)")
```



Interpretation:

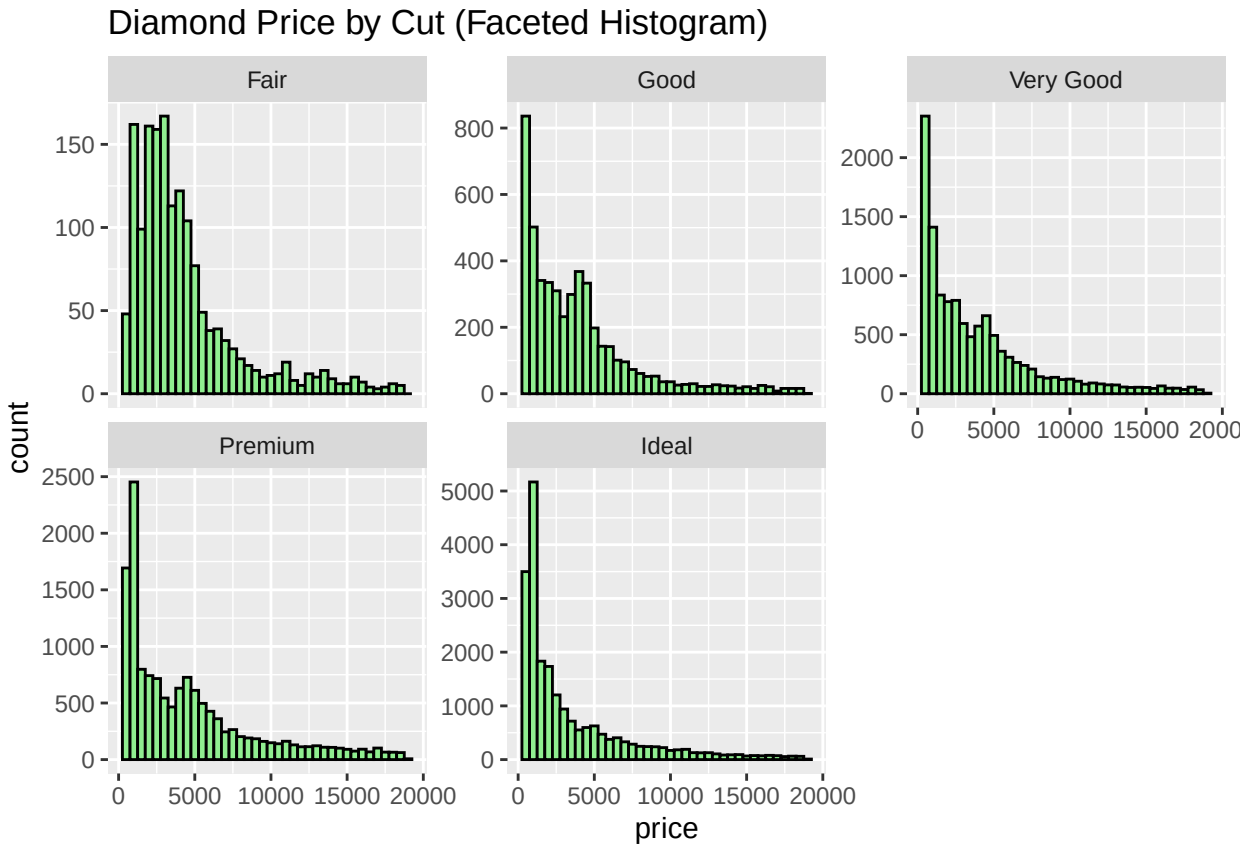
- Each "violin" shows the full distribution of prices per cut level — wide parts mean many observations.
- Symmetrical shape combines boxplot + kernel density.
- Shows both spread and density.

(b) Faceted histogram

- **Pros:**
  - Displays the full distribution separately for each category, avoiding overlap.
  - Easy to see skewness, peaks, and differences in shape among groups.

- **Cons:**
  - Requires more space and may be harder to compare across facets.
  - If `scales = "free_y"` is used, absolute frequencies become hard to compare.

```r
ggplot(diamonds, aes(x = price)) +
  geom_histogram(binwidth = 500, fill = "lightgreen", color = "black") +
  facet_wrap(~cut, scales = "free_y") +
  labs(title = "Diamond Price by Cut (Faceted Histogram)")
```



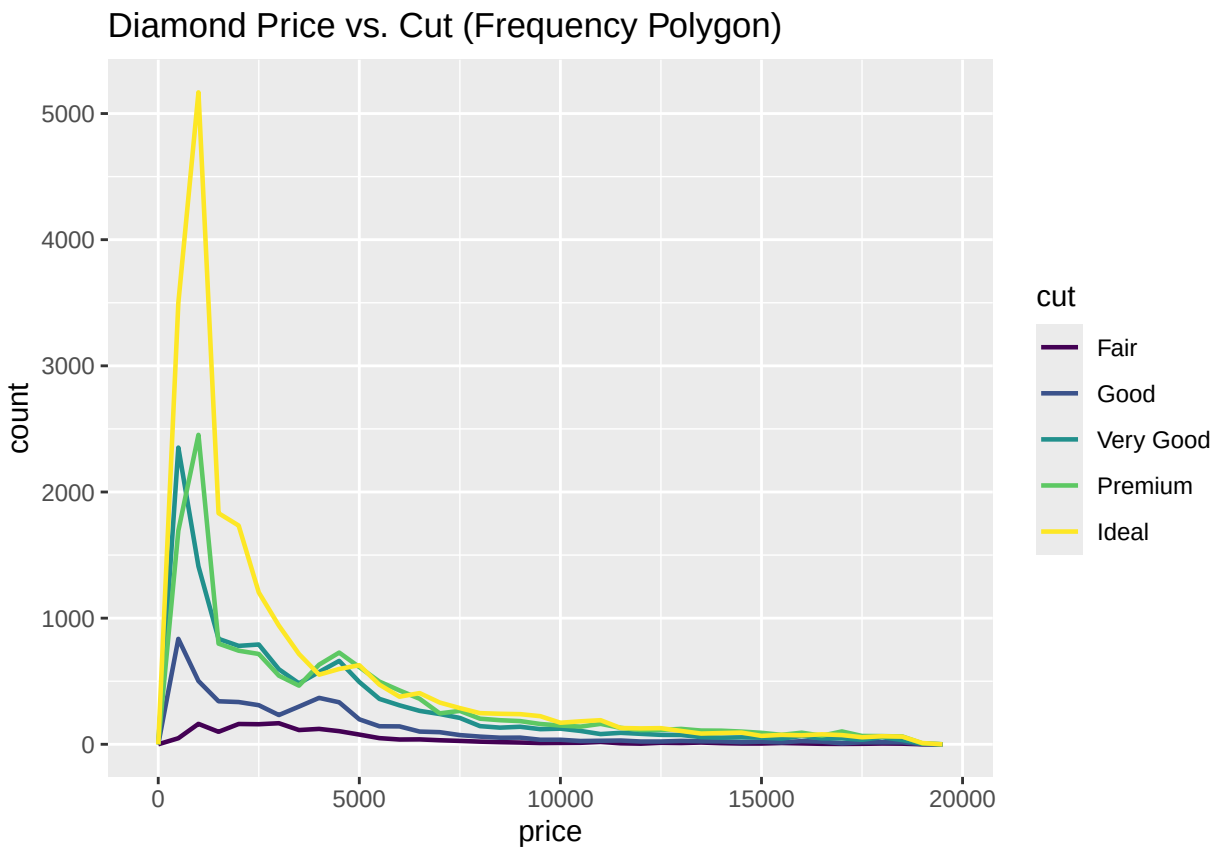
Interpretation:

- Separate histogram for each cut category.
- Makes it easier to see distribution shape per group.
- scales = "free_y" allows different count scales so patterns don't get squished.

(c) Colored frequency polygon

- **Pros:**
  - Effective for comparing multiple distributions on a single plot.
  - Easier to read general trends compared to stacked histograms.
- **Cons:**
  - Overlapping lines can become cluttered and confusing.
  - Sensitive to binwidth choice, which affects shape perception.

```r
ggplot(diamonds, aes(x = price, color = cut)) +
  geom_freqpoly(binwidth = 500, linewidth = 0.8) +
  labs(title = "Diamond Price vs. Cut (Frequency Polygon)")
```



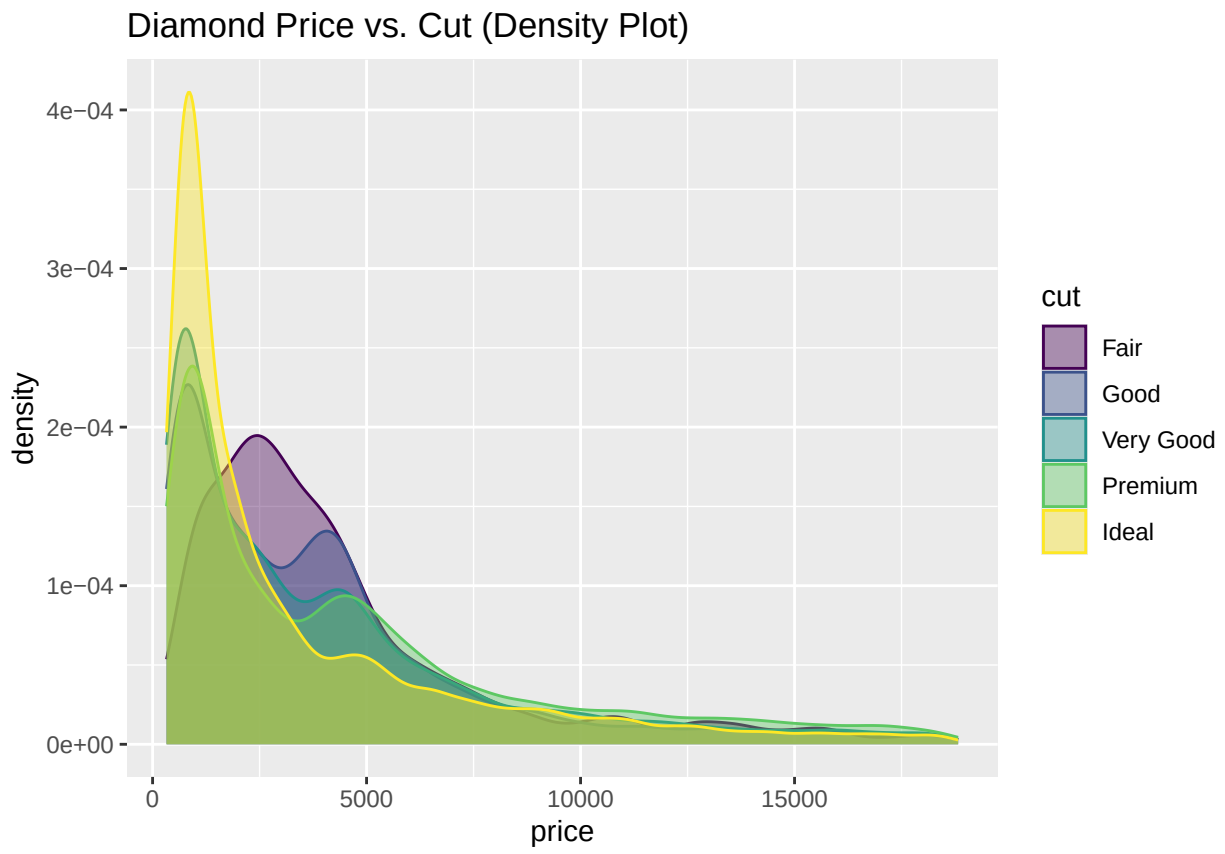
Interpretation:

- Overlaid line plots (one per cut) showing counts per price range.
- Highlights comparative shape differences between groups.
- Can get cluttered when distributions overlap a lot.

(d) Colored density plot

- **Pros:**
  - Smooth representation of distribution shapes for each group.
  - Normalized by area, allowing fair comparison of differently sized groups.
- **Cons:**
  - Loses count (frequency) information due to normalization.
  - Overlapping colors can make individual group patterns harder to distinguish.

```
ggplot(diamonds, aes(x = price, fill = cut, color = cut)) +
  geom_density(alpha = 0.4) +
  labs(title = "Diamond Price vs. Cut (Density Plot)")
```



Interpretation:

- Smooth curve representing the probability density of prices per cut.
- Alpha blending allows overlapping regions to be seen.
- Good for overall trend comparison without worrying about binwidth.

Section 10.5.1.1: Problem 6

Q) If you have a small dataset, it's sometimes useful to use geom_jitter() to avoid overplotting to more easily see the relationship between a continuous and categorical variable. The ggbeeswarm package provides a number of methods similar to geom_jitter(). List them and briefly describe what each one does.

Ans:

Methods from ggbeeswarm package

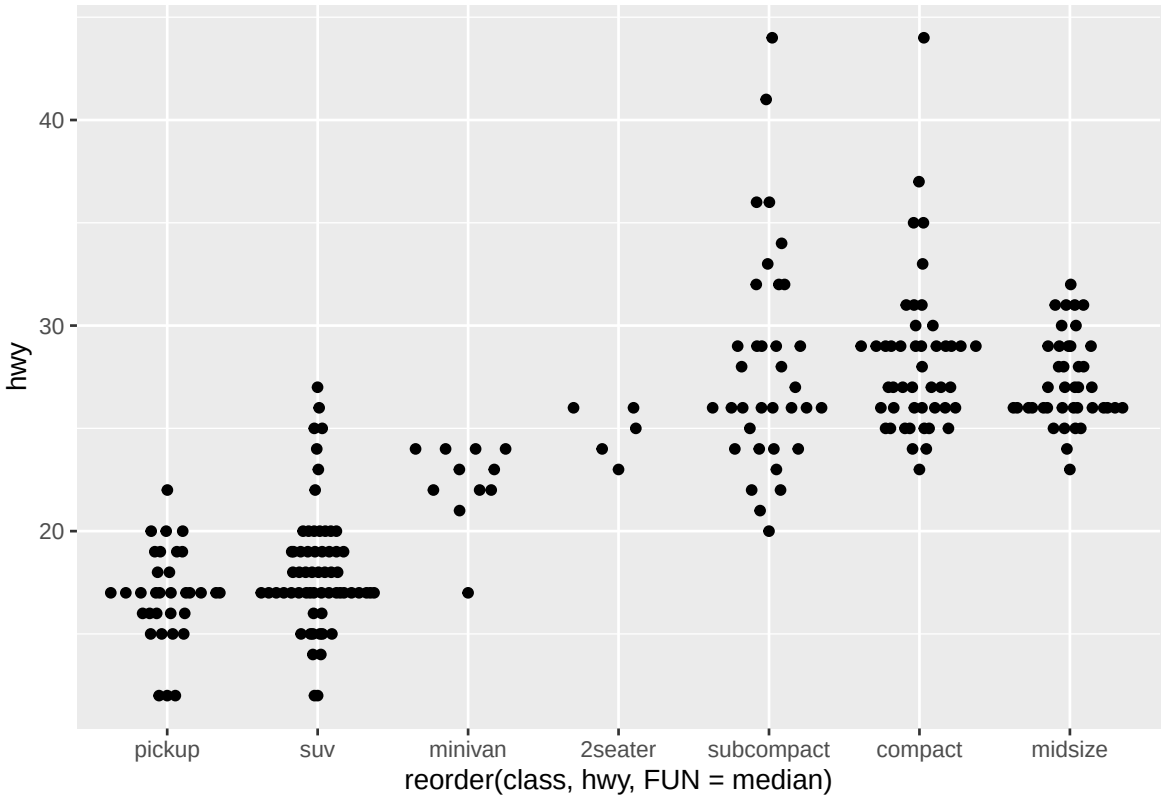
```
library(ggbeeswarm)
```

1. **geom_quasirandom()**
    - Produces plots that combine the features of jitter and violin plots.

- Randomly spreads out the data points along the x-axis while maintaining the general shape of the data distribution.

- Example:

```
ggplot(data = mpg) +
  geom_quasirandom(mapping = aes(
    x = reorder(class, hwy, FUN = median),
    y = hwy
  ))
```



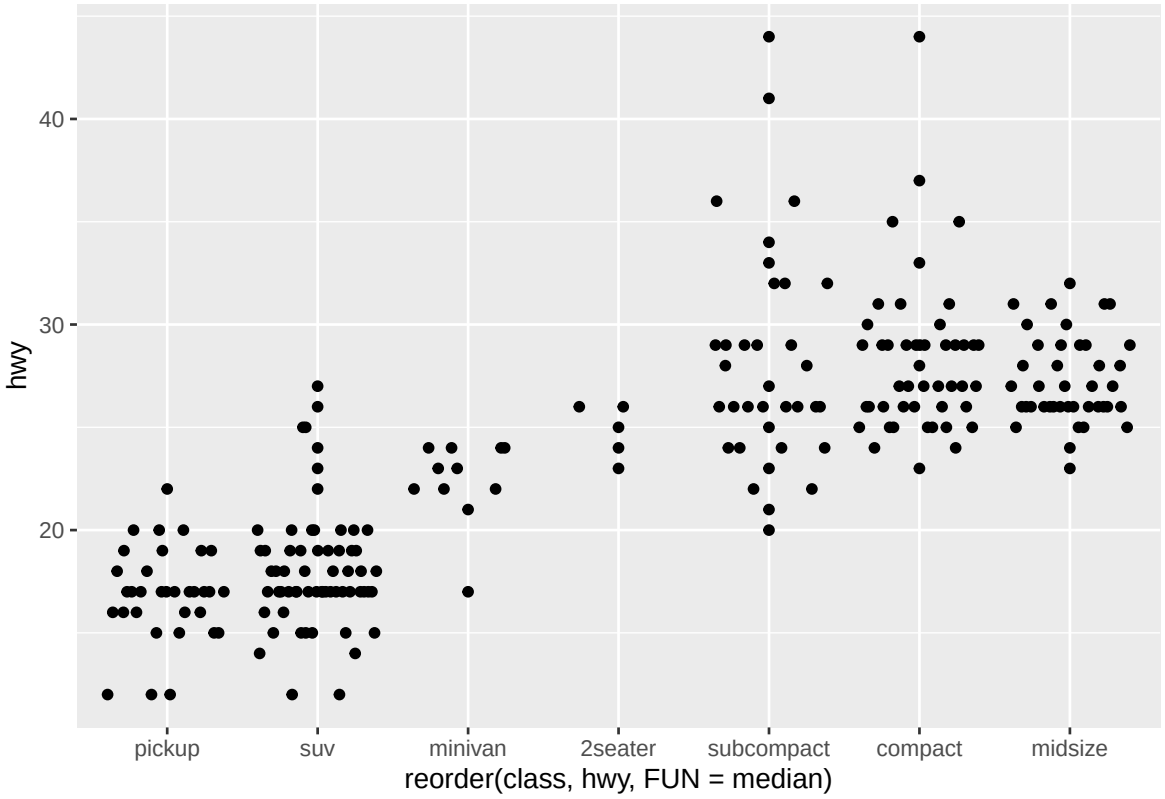
2. **geom_quasirandom(method = "tukey")**

- Uses the *Tukey* method to determine how the points are spread.

- This method spaces the points based on density but tends to create a cleaner, more symmetric distribution.

- Example:

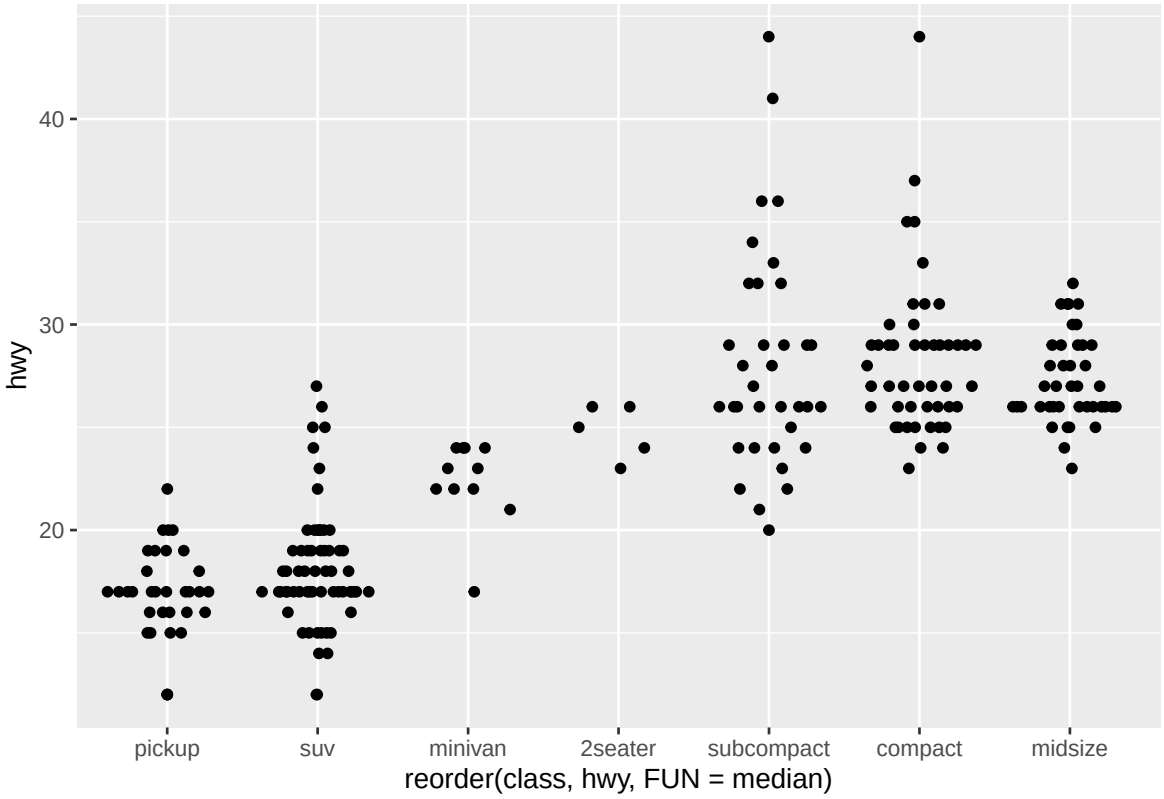
```
ggplot(data = mpg) +
  geom_quasirandom(
    mapping = aes(
      x = reorder(class, hwy, FUN = median),
      y = hwy
    ),
    method = "tukey"
  )
```



3. **geom_quasirandom(method = "tukeyDense")**

   - A denser version of the Tukey method.
   - Points are packed closer together for compact visualization, useful when data points are numerous.

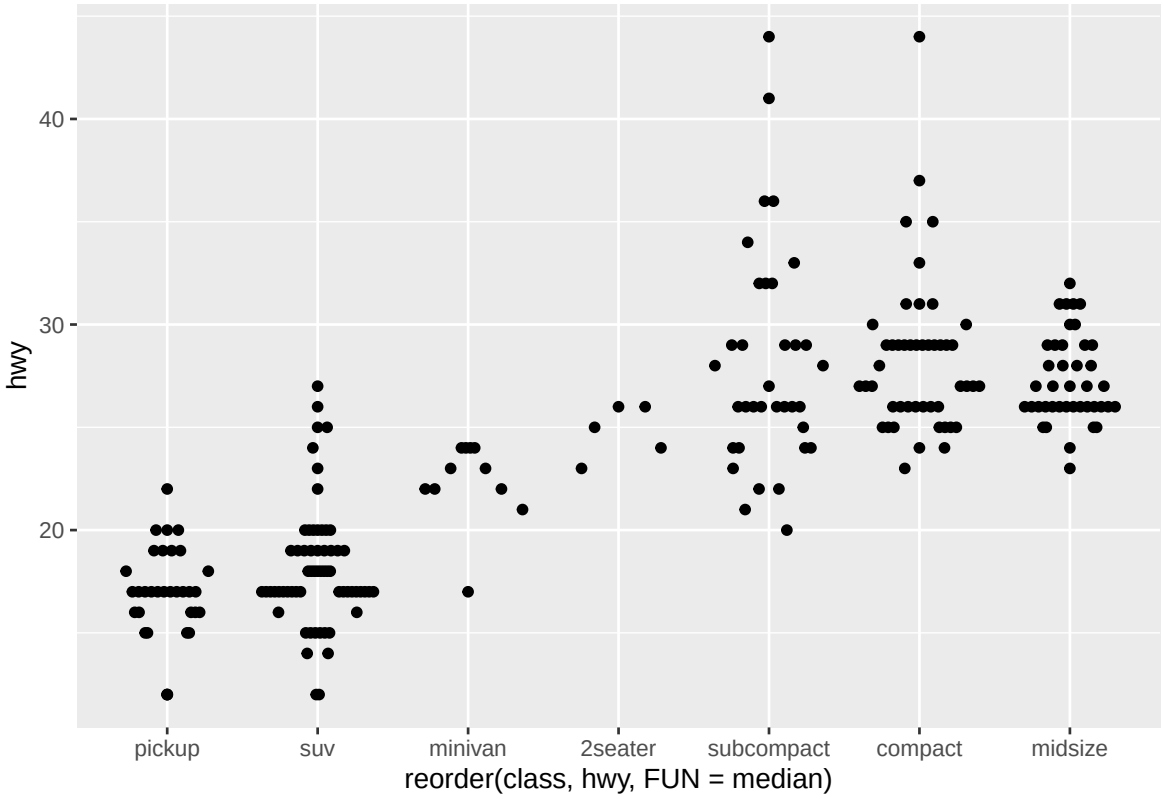
```
ggplot(data = mpg) +
  geom_quasirandom(
    mapping = aes(
      x = reorder(class, hwy, FUN = median),
      y = hwy
    ),
    method = "tukeyDense"
  )
```



4. **geom_quasirandom(method = "frowney")**

   - Arranges the points in a slightly "frowning" arc pattern.
   - Helps differentiate overlapping points while giving a visually distinctive shape.
   - Example:

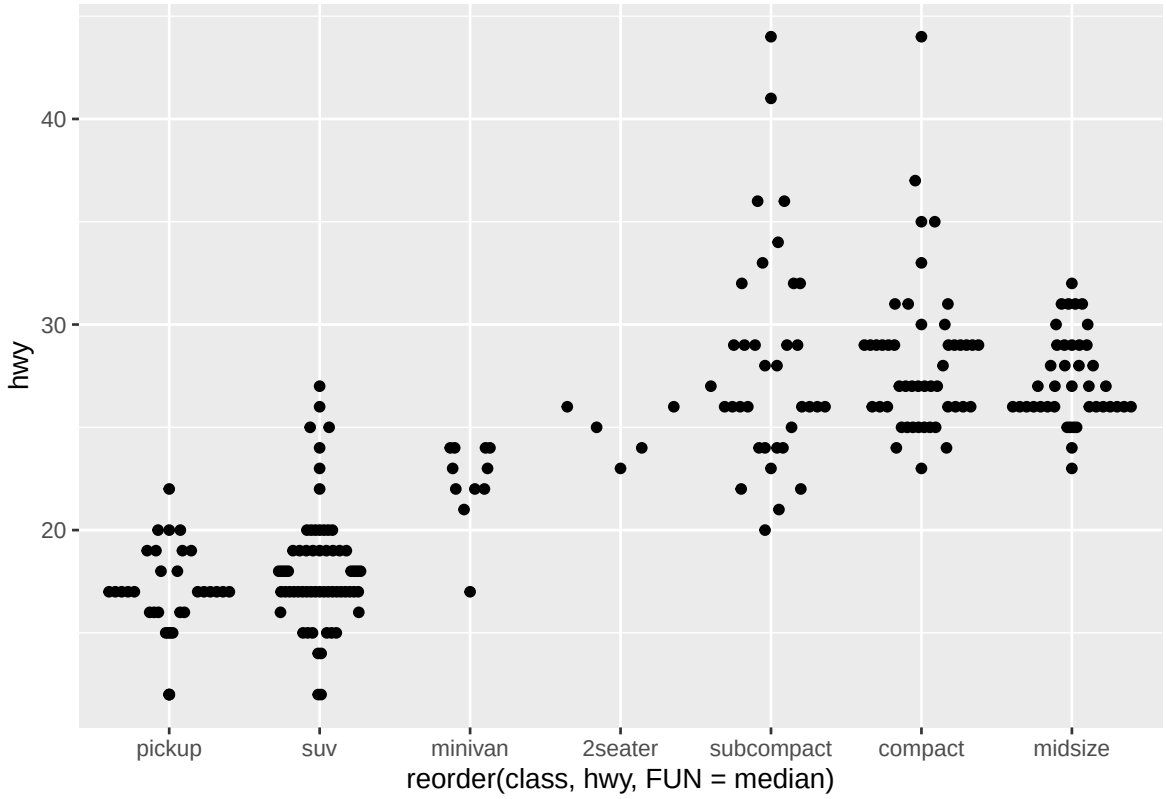
```
ggplot(data = mpg) +
  geom_quasirandom(
    mapping = aes(
      x = reorder(class, hwy, FUN = median),
      y = hwy
    ),
    method = "frowney"
  )
```



5. **geom_quasirandom(method = "smiley")**

   - Arranges the points in a "smiling" arc shape.
   - Similar to "frowney" but curved upward, giving a different visual style while preserving distribution insight.
   - Example:

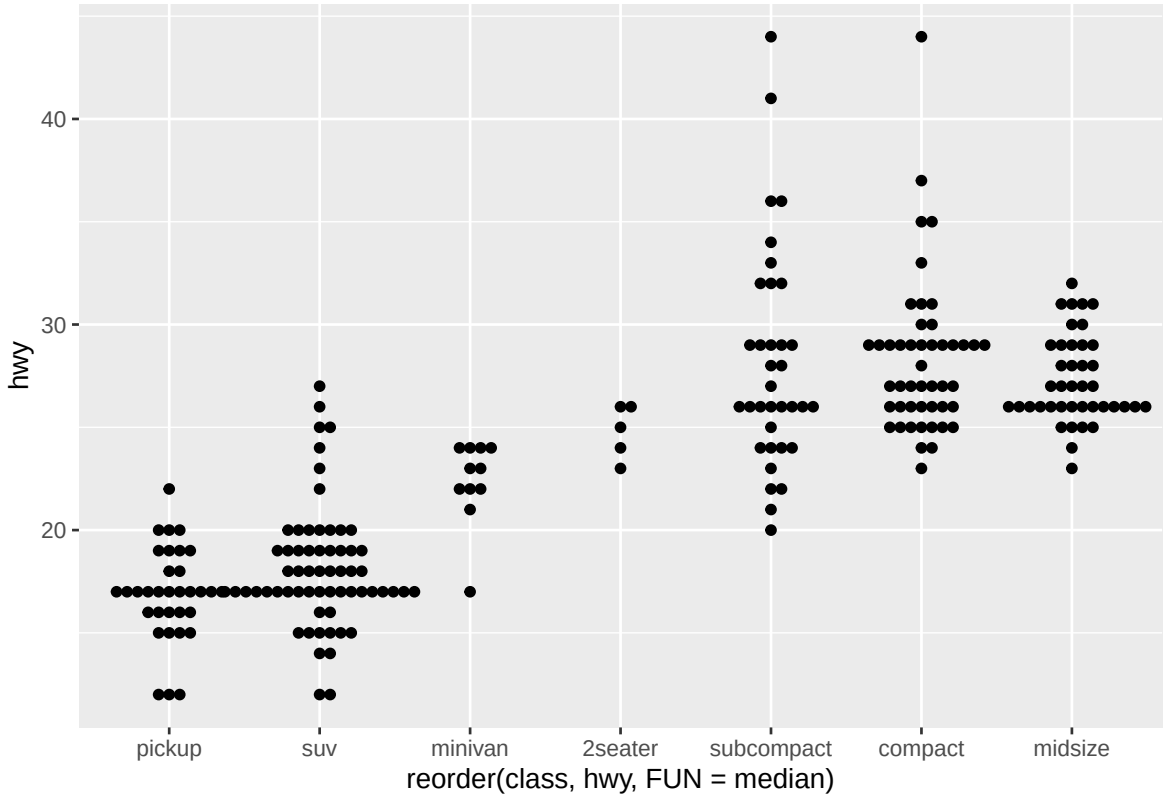
```
ggplot(data = mpg) +
  geom_quasirandom(
    mapping = aes(
      x = reorder(class, hwy, FUN = median),
      y = hwy
    ),
    method = "smiley"
  )
```



6. **geom_beeswarm()**

   - Produces a beeswarm plot where points are systematically offset to prevent overlap.
   - Creates a plot that looks similar to a violin plot but emphasizes individual data points rather than density.
   - Example:

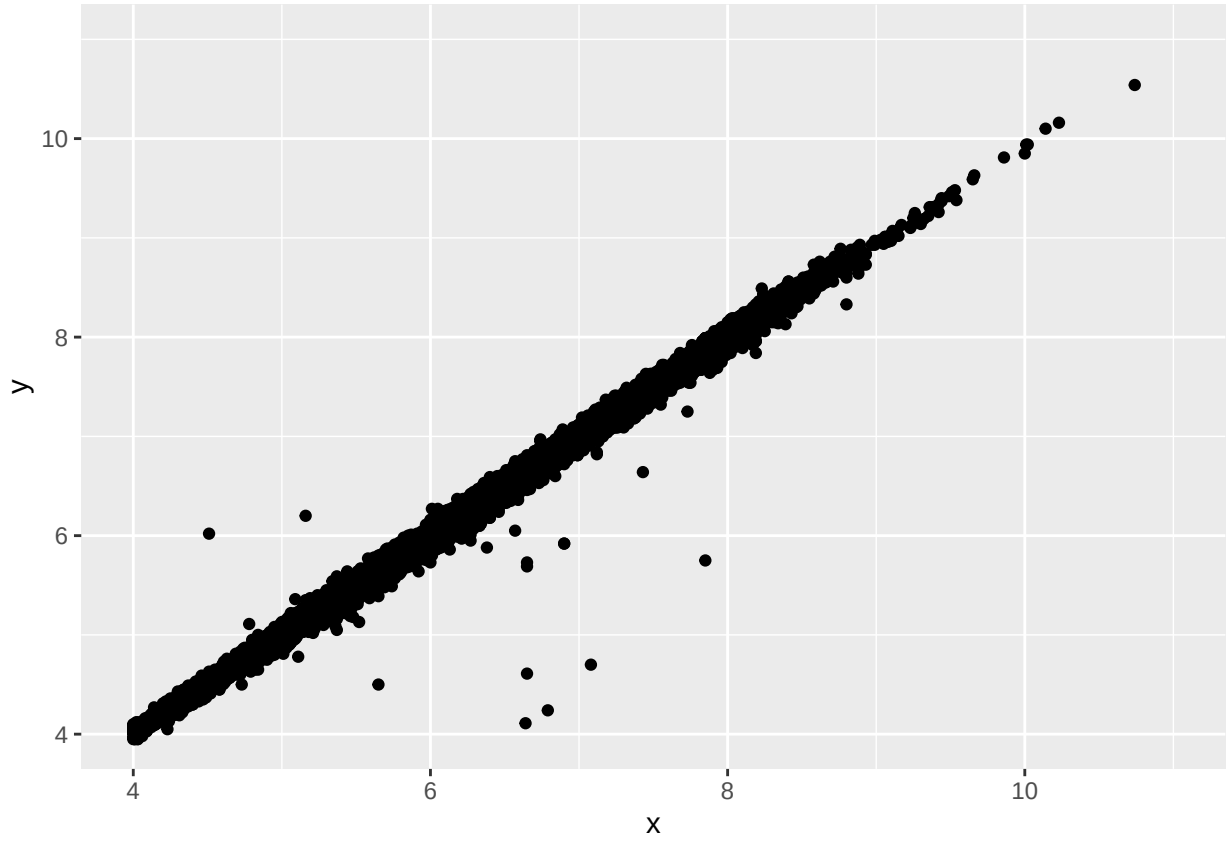
```
ggplot(data = mpg) +
  geom_beeswarm(mapping = aes(
    x = reorder(class, hwy, FUN = median),
    y = hwy
  ))
```



Section 10.5.3.1: Problem 5

Q) Two dimensional plots reveal outliers that are not visible in one dimensional plots. For example, some points in the following plot have an unusual combination of x and y values, which makes the points outliers even though their x and y values appear normal when examined separately. Why is a scatterplot a better display than a binned plot for this case?

```
diamonds |>
  filter(x >= 4) |>
  ggplot(aes(x = x, y = y)) +
  geom_point() +
  coord_cartesian(xlim = c(4, 11), ylim = c(4, 11))
```



Ans:

A **scatterplot** is a better display than a **binned plot** in this case because:

- A scatterplot shows each individual data point, allowing you to clearly identify unusual combinations of `x` and `y` values (true outliers in two dimensions).
- In contrast, a binned plot (like `geom_bin2d()` or `geom_hex()`) aggregates points into bins, which can hide outliers by averaging them with nearby points—making it harder to detect single, abnormal observations.

Section 11.4.6: Problem 3

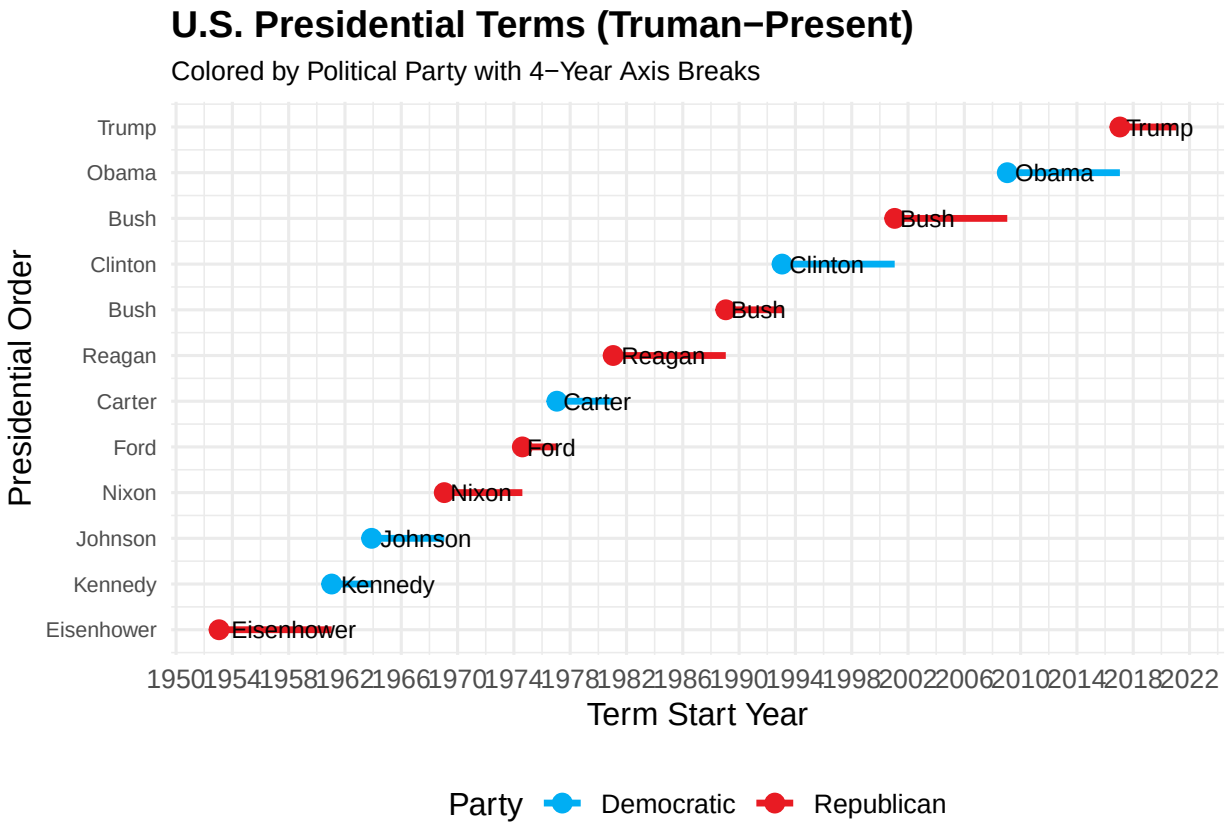
Q) Change the display of the presidential terms by:

a. Combining the two variants that customize colors and x axis breaks.
b. Improving the display of the y axis.
c. Labelling each term with the name of the president.
d. Adding informative plot labels.
e. Placing breaks every 4 years (this is trickier than it seems!).

Ans:

```
### Answer
```

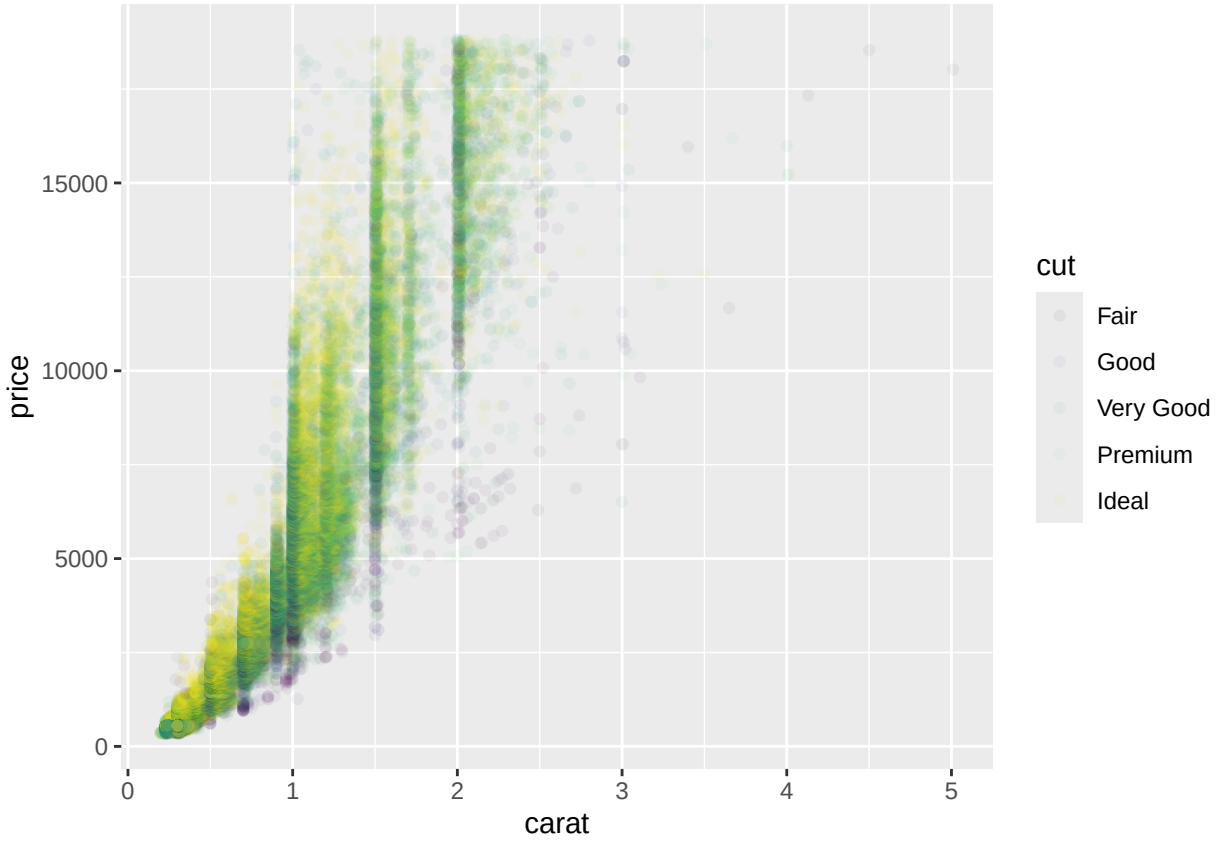
```r
# Load necessary packages
library(tidyverse)
library(lubridate)

# Improved visualization of presidential terms
presidential |>
  mutate(id = 33 + row_number()) |>
  ggplot(aes(x = start, y = id, color = party)) +
  geom_point(size = 3) +
  geom_segment(aes(xend = end, yend = id), linewidth = 1.2) +
  geom_text(aes(label = name), hjust = -0.1, size = 3.2, color = "black") +
  scale_color_manual(
    values = c(Republican = "#E81B23", Democratic = "#00AEF3"),
    name = "Party"
  ) +
  scale_x_date(
    name = "Term Start Year",
    breaks = seq(ymd("1950-01-01"), ymd("2025-01-01"), by = "4 years"),
    date_labels = "%Y"
  ) +
  scale_y_continuous(
    name = "Presidential Order",
    breaks = 33 + seq_len(nrow(presidential)),
    labels = presidential$name
  ) +
  labs(
    title = "U.S. Presidential Terms (Truman-Present)",
    subtitle = "Colored by Political Party with 4-Year Axis Breaks",
    x = "Year",
    y = "President",
    color = "Political Party"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",
    axis.text.y = element_text(size = 8),
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 10)
  )
```



Section 11.4.6: Problem 4

Q) First, create the following plot. Then, modify the code using override.aes to make the legend easier to see.

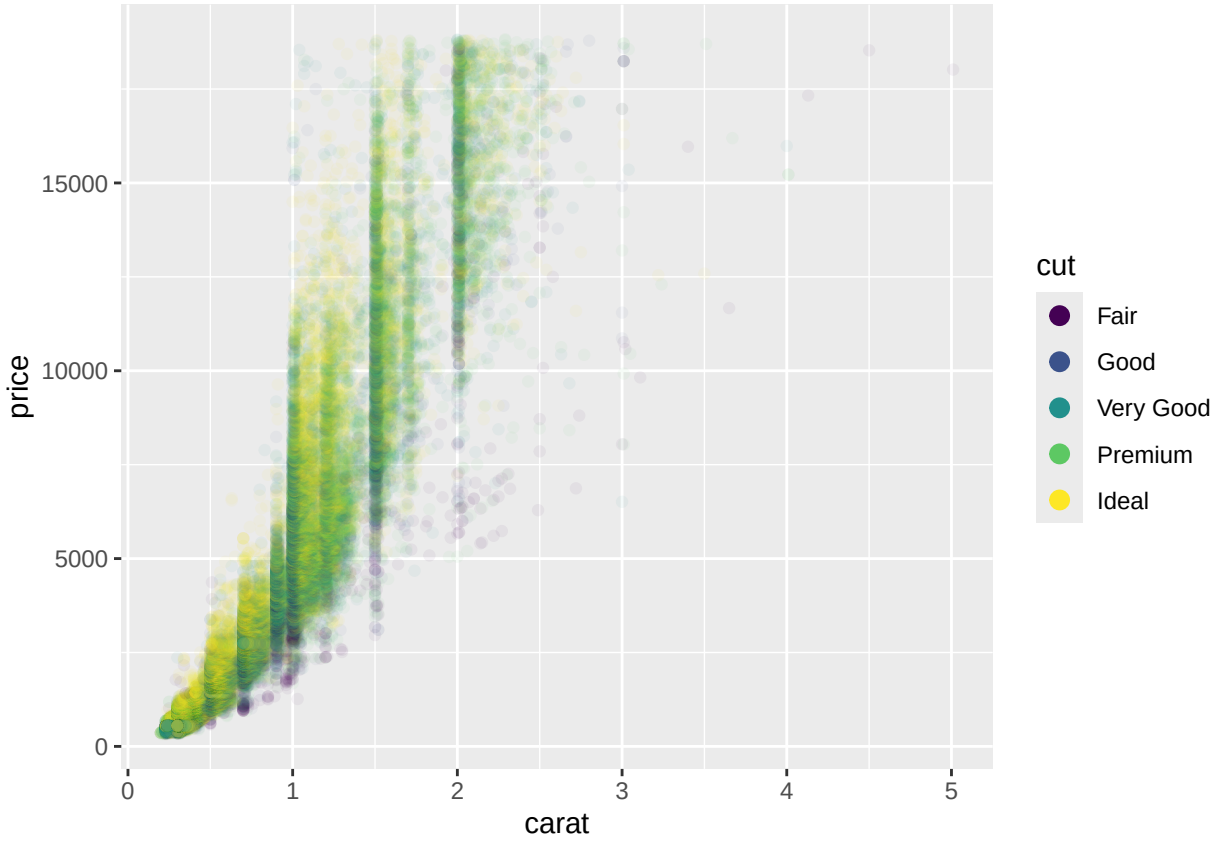
```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_point(aes(color = cut), alpha = 1/20)
```



Ans:

Below is the modified version of your code using override.aes to make the legend easier to see:

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_point(aes(color = cut), alpha = 1/20) +
  guides(color = guide_legend(override.aes = list(alpha = 1, size = 3)))
```



Explanation:

- `guides()` is used to customize legends in ggplot2.
- `guide_legend()` specifically modifies legends for categorical variables (like cut).
- `override.aes` allows you to change aesthetics only in the legend:
  - `alpha = 1` makes the legend symbols fully opaque.
  - `size = 3` increases the legend point size for better visibility.

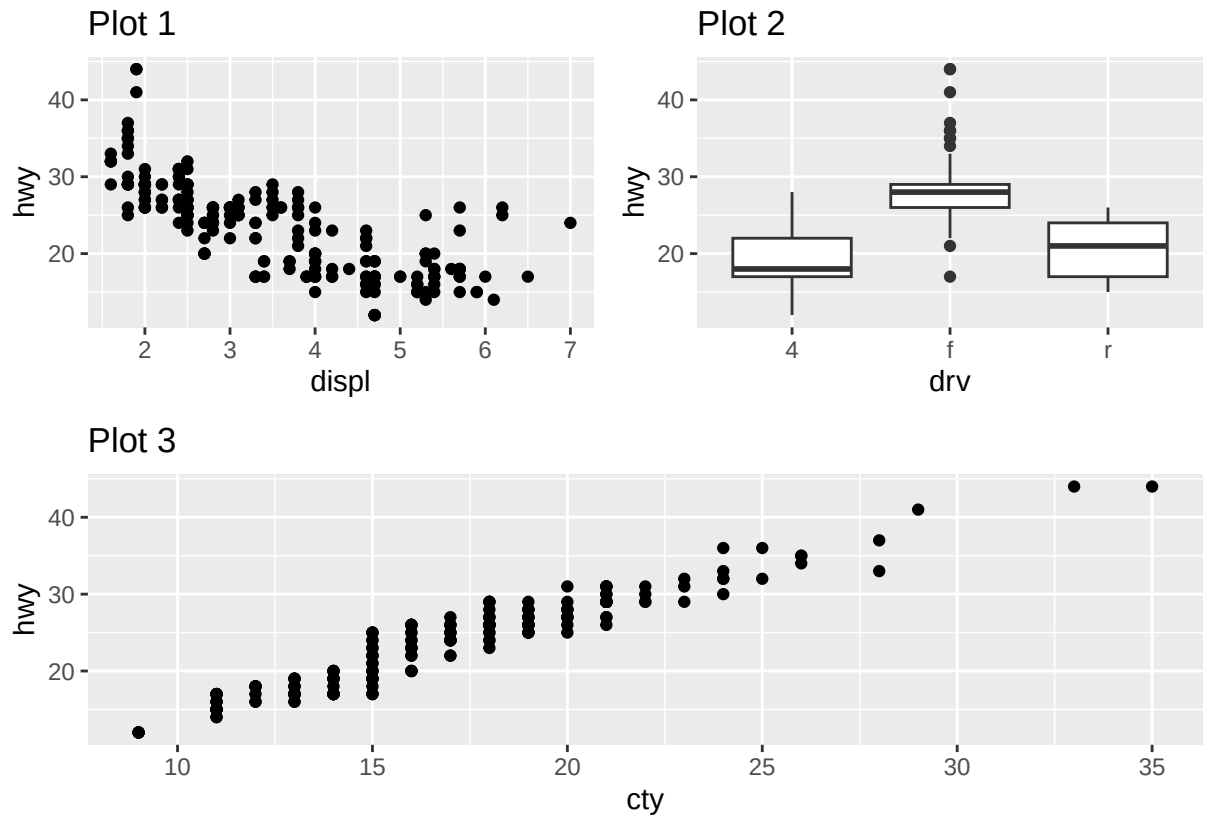
Section 11.6.1: Problem 1

Q) What happens if you omit the parentheses in the following plot layout. Can you explain why this happens?

```
library(ggplot2)
library(patchwork)
```

```
p1 <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(title = "Plot 1")
p2 <- ggplot(mpg, aes(x = drv, y = hwy)) +
  geom_boxplot() +
  labs(title = "Plot 2")
p3 <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() +
```

```
  labs(title = "Plot 3")

(p1 | p2) / p3
```



Ans:

Effect of Omitting Parentheses in Patchwork Layouts

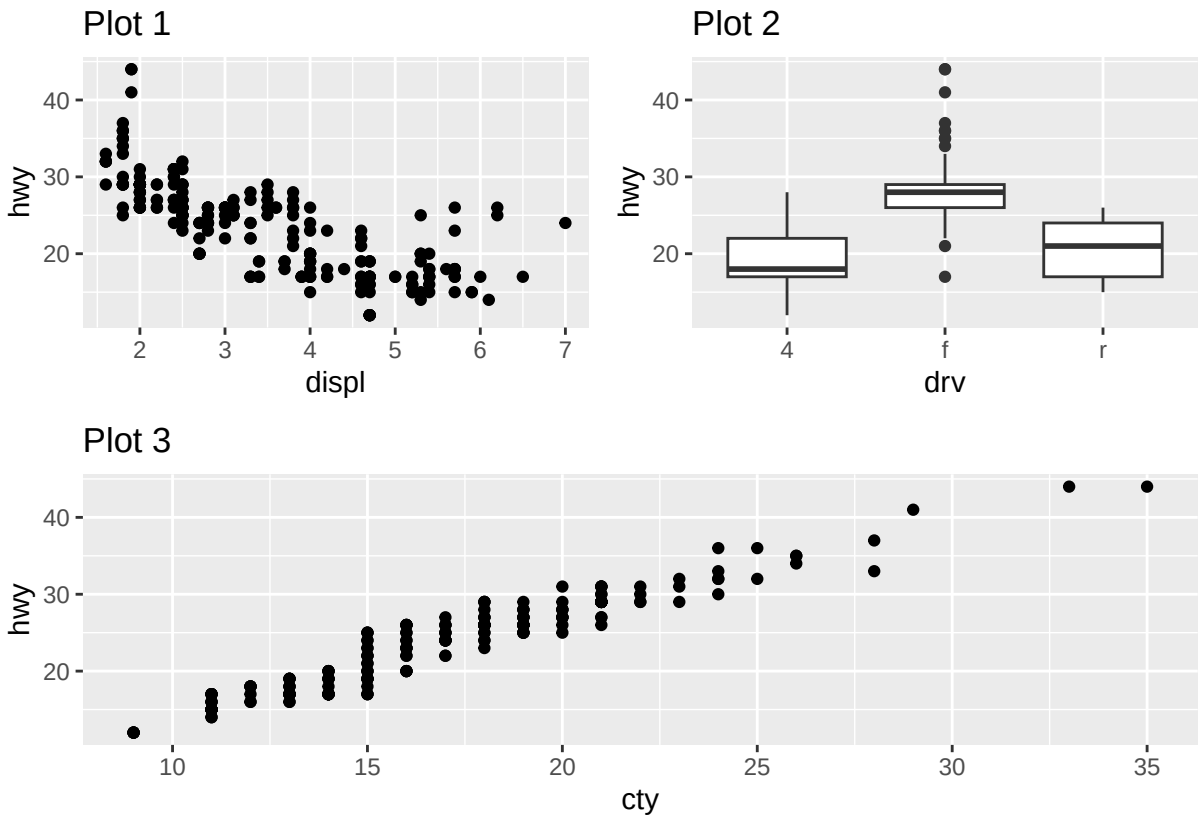
The patchwork package uses operators like | and / to arrange multiple ggplots.
- | places plots **side by side** (horizontally).
- / places plots **on top of each other** (vertically).

Parentheses determine **the order of operations** for layout composition — just like in arithmetic.

```
library(ggplot2)
library(patchwork)
```

```
# Create three example plots
p1 <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(title = "Plot 1")

p2 <- ggplot(mpg, aes(x = drv, y = hwy)) +
  geom_boxplot() +
  labs(title = "Plot 2")
```

```
p3 <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() +
  labs(title = "Plot 3")

# Correct layout using parentheses
(p1 | p2) / p3
```



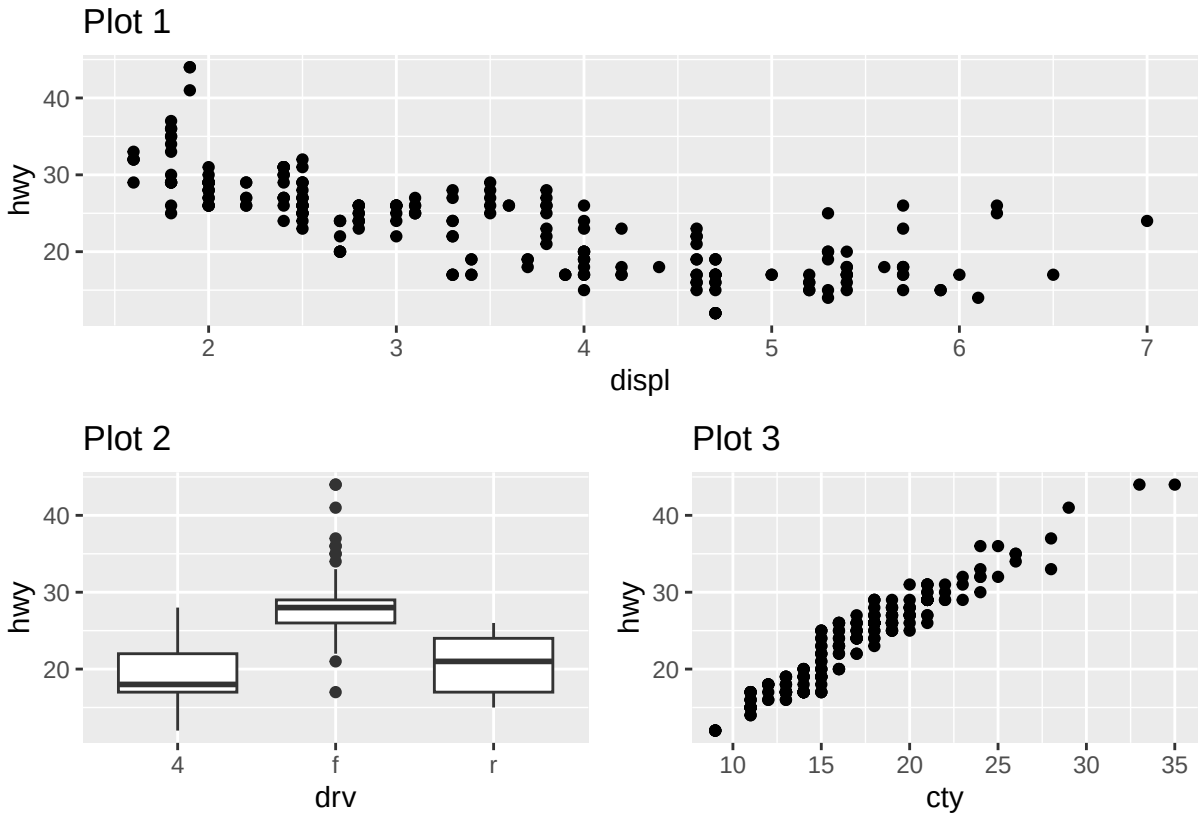
Section 11.6.1: Problem 2

Q) Using the three plots from the previous exercise, recreate the following patchwork.

Ans: ### Recreating the Patchwork Layout

```
library(ggplot2)
library(patchwork)

# Plot 1: Scatterplot of highway vs engine displacement
p1 <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(title = "Plot 1")

# Plot 2: Boxplot of highway mileage by drive type
p2 <- ggplot(mpg, aes(x = drv, y = hwy)) +
```

```
  geom_boxplot() +
  labs(title = "Plot 2")

# Plot 3: Scatterplot of highway vs city mileage
p3 <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() +
  labs(title = "Plot 3")

# Combine plots using patchwork
# Plot 1 on top, Plot 2 and Plot 3 side by side underneath
p1 / (p2 | p3)
```



Problems from R ISLRv book:

Section 5.4: Problem 2

Q) We will now derive the probability that a given observation is part of a bootstrap sample. Suppose that we obtain a bootstrap sample from a set of n observations.

Ans:

(a) What is the probability that the first bootstrap observation is *not* the jth observation from the original sample? Justify your answer.

- This is 1 - probability that it is the jth $= 1 - 1/n$.

(b) What is the probability that the second bootstrap observation is not the jth observation from the original sample?

- Since each bootstrap observation is a random sample, this probability is the same $(1 - 1/n)$.

(c) Argue that the probability that the jth observation is not in the bootstrap sample is (1- 1/n)n.

- For the jth observation to not be in the sample, it would have to *not* be picked for each of n positions, so not picked for $1, 2, ..., n$, thus the probability is $(1 - 1/n)^n$

(d) When $n = 5$, what is the probability that the jth observation is in the bootstrap sample?

- $p = 0.67$

```
n <- 5
1 - (1 - 1 / n)^n
```

```
## [1] 0.67232
```

(e) When $n = 100$, what is the probability that the jth observation is in the bootstrap sample?

- $p = 0.64$

```
n <- 100
1 - (1 - 1 / n)^n
```

```
## [1] 0.6339677
```

(f) When $n = 10,000$, what is the probability that the jth observation is in the bootstrap sample?

- $p = 0.63$

```
n <- 100000
1 - (1 - 1 / n)^n
```

```
## [1] 0.6321224
```

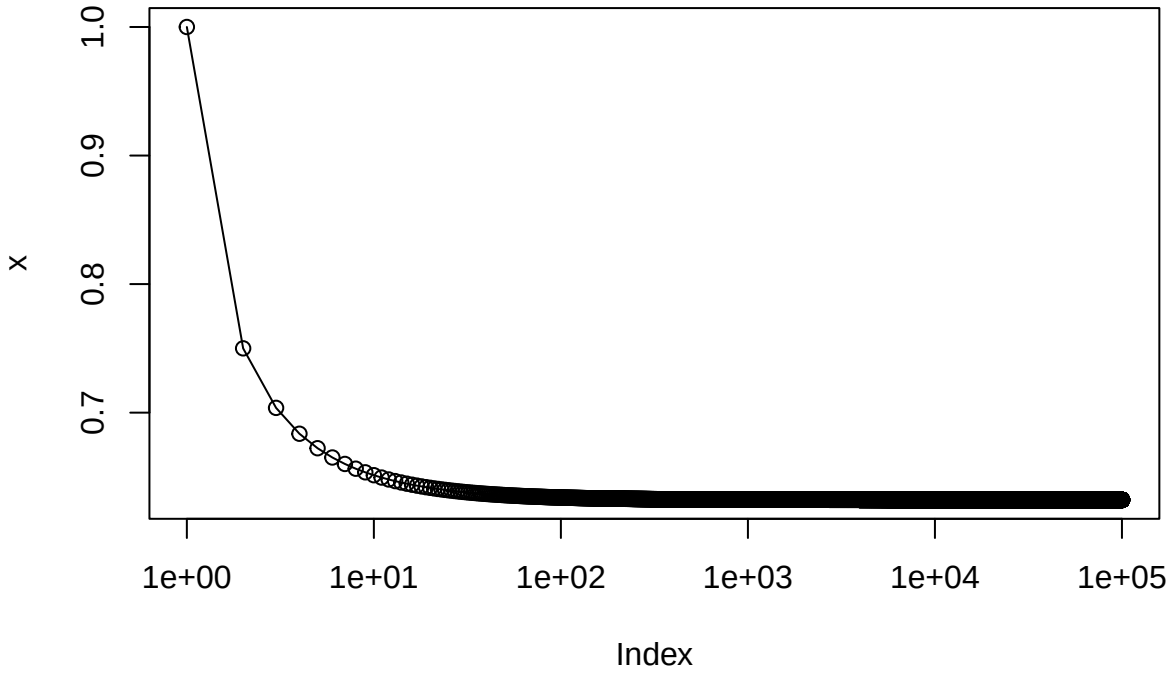
(g) Create a plot that displays, for each integer value of n from 1 to 100,000, the probability that the jth observation is in the bootstrap sample. Comment on what you observe.

- The probability rapidly approaches 0.63 with increasing n.
- Note that

$$e^x = \lim_{x \to \text{inf}} \left(1 + \frac{x}{n}\right)^n,$$

so with $x = -1$, we can see that our limit is $1 - e^{-1} = 1 - 1/e$.

```
x <- sapply(1:100000, function(n) 1 - (1 - 1 / n)^n)
plot(x, log = "x", type = "o")
```



(h) We will now investigate numerically the probability that a bootstrap sample of size $n = 100$ contains the jth observation. Here $j = 4$. We repeatedly create bootstrap samples, and each time we record whether or not the fourth observation is contained in the bootstrap sample.

- The probability of including 4 when resampling numbers 1...100 is close to $1 - (1 - 1/100)^{100}$.

```
store <- rep (NA, 10000)
for (i in 1:10000) {
store[i] <- sum(sample(1:100, rep = TRUE) == 4) > 0
}
mean(store)
```

```
## [1] 0.6395
```

```
store <- replicate(10000, sum(sample(1:100, replace = TRUE) == 4) > 0)
mean(store)
```

```
## [1] 0.6334
```

Section 5.4: Problem 9

Q) We will now consider the Boston housing data set, from the ISLR2 library.

Ans:

(a) Based on this data set, provide an estimate for the population mean of `medv`. Call this estimate $\hat{\mu}$.

```
data("Boston", package = "ISLR2")
```

```
(mu <- mean(Boston$medv))
```

```
## [1] 22.53281
```

(b) Provide an estimate of the standard error of $\hat{\mu}$. Interpret this result.

Hint: We can compute the standard error of the sample mean by dividing the sample standard deviation by the square root of the number of observations.

```
sd(Boston$medv) / sqrt(length(Boston$medv))
```

```
## [1] 0.4088611
```

(c) Now estimate the standard error of $\hat{\mu}$ using the bootstrap. How does this compare to your answer from (b)?

- The standard error using the bootstrap (0.403) is very close to that obtained from the formula above (0.409).

```
library(ISLR2)
library(boot)
```

```
set.seed(42)
(bs <- boot(Boston$medv, function(v, i) mean(v[i]), 10000))
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Boston$medv, statistic = function(v, i) mean(v[i]),
##     R = 10000)
##
##
## Bootstrap Statistics :
##     original      bias    std. error
## t1* 22.53281 0.002175751   0.4029139
```

(d) Based on your bootstrap estimate from (c), provide a 95% confidence interval for the mean of `medv`. Compare it to the results obtained using `t.test(Boston$medv)`.

Hint: You can approximate a 95% confidence interval using the formula $[\hat{\mu} - 2SE(\hat{\mu}), \hat{\mu} + 2SE(\hat{\mu})]$.

```r
se <- sd(bs$t)
c(mu - 2 * se, mu + 2 * se)
```

```
## [1] 21.72698 23.33863
```

(e) Based on this data set, provide an estimate, $\hat{\mu}_{med}$, for the median value of `medv` in the population.

```r
median(Boston$medv)
```

```
## [1] 21.2
```

(f) We now would like to estimate the standard error of $\hat{\mu}_{med}$. Unfortunately, there is no simple formula for computing the standard error of the median. Instead, estimate the standard error of the median using the bootstrap. Comment on your findings.

- The estimated standard error of the median is 0.374. This is lower than the standard error of the mean.

```r
set.seed(42)
boot(Boston$medv, function(v, i) median(v[i]), 10000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Boston$medv, statistic = function(v, i) median(v[i]),
##     R = 10000)
##
##
## Bootstrap Statistics :
##     original   bias    std. error
## t1*     21.2 -0.01331   0.3744634
```

(g) Based on this data set, provide an estimate for the tenth percentile of `medv` in Boston census tracts. Call this quantity $\hat{\mu}_{0.1}$. (You can use the `quantile()` function.)

```r
quantile(Boston$medv, 0.1)
```

```
##   10%
## 12.75
```

(h) Use the bootstrap to estimate the standard error of $\hat{\mu}_{0.1}$. Comment on your findings.

- We get a standard error of ~0.5. This is higher than the standard error of the median. Nevertheless the standard error is quite small, thus we can be fairly confidence about the value of the 10th percentile.

```r
set.seed(42)
boot(Boston$medv, function(v, i) quantile(v[i], 0.1), 10000)
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Boston$medv, statistic = function(v, i) quantile(v[i],
##     0.1), R = 10000)
##
##
## Bootstrap Statistics :
##     original   bias    std. error
## t1*    12.75 0.013405    0.497298
```

Section 6.6: Problem 1

Q) We perform best subset, forward stepwise, and backward stepwise selection on a single data set. For each approach, we obtain $p + 1$ models, containing $0, 1, 2, ..., p$ predictors. Explain your answers:

(a) Which of the three models with k predictors has the smallest *training* RSS?

Ans: Best subset considers the most models (all possible combinations of p predictors are considered), therefore this will give the smallest training RSS (it will at least consider all possibilities covered by forward and backward stepwise selection). However, all three approaches are expected to give similar if not identical results in practice.

(b) Which of the three models with k predictors has the smallest *test* RSS?

Ans: We cannot tell which model will perform best on the test RSS. The answer will depend on the tradeoff between fitting to the data and overfitting.

(c) True or False:

```
   (i) The predictors in the $k$-variable model identified by forward stepwise are a subset of the pred

Ans: True. Forward stepwise selection retains all features identified in previous models as $k$ is incr

(ii) The predictors in the $k$-variable model identified by backward stepwise are a subset of the predi

Ans: True. Backward stepwise selection removes features one by one as $k$ is decreased.

(iii) The predictors in the $k$-variable model identified by backward stepwise are a subset of the pred

Ans: False. Forward and backward stepwise selection can identify different combinations of variables du

(iv) The predictors in the $k$-variable model identified by forward stepwise are a subset of the predic

False. Forward and backward stepwise selection can identify different combinations of variables due to

(v)  The predictors in the $k$-variable model identified by best subset are a subset of the predictors

False. Best subset selection can identify different combinations of variables for each $k$ by consideri
```

Section 6.6: Problem 11

Q) We will now try to predict per capita crime rate in the `Boston` data set.

Ans:

```
library(ISLR2)
library(leaps)
library(glmnet)
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

```
library(pls)
```

```
##
## Attaching package: 'pls'
```

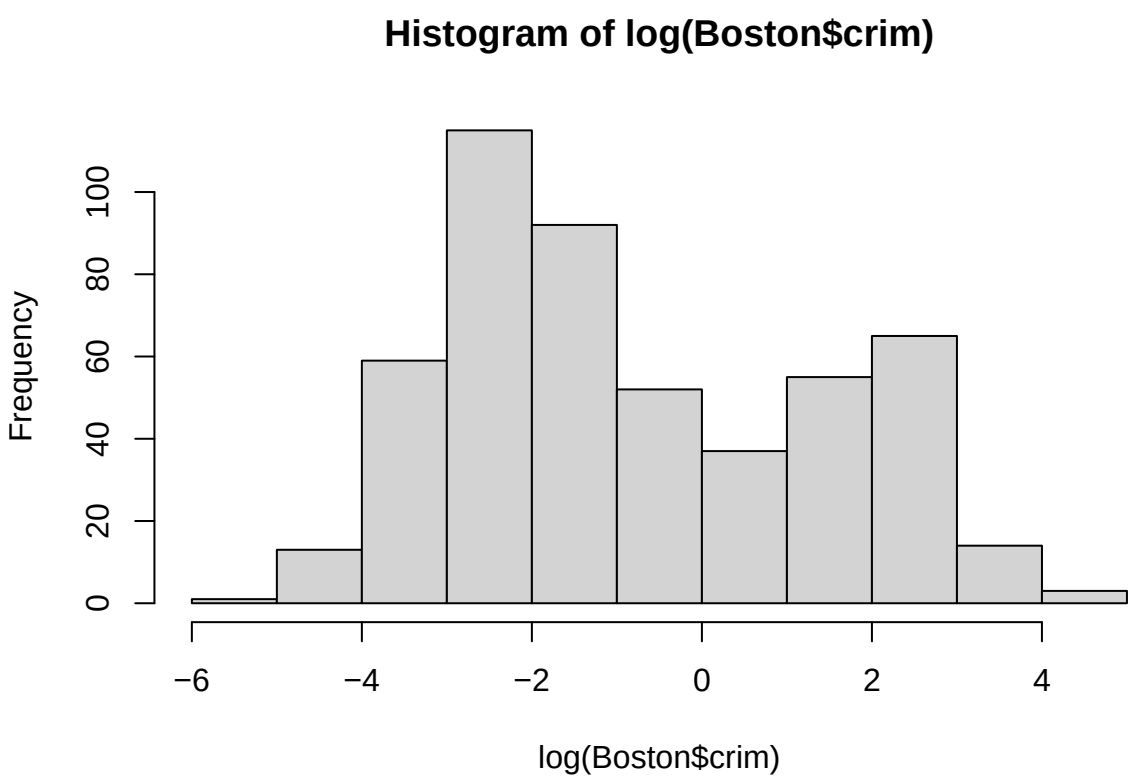
```
## The following object is masked from 'package:stats':
##
##     loadings
```

```
library(boot)
```

a. Try out some of the regression methods explored in this chapter, such as > best subset selection, the lasso, ridge regression, and PCR. Present and > discuss results for the approaches that you consider.

Ans:

```
set.seed(1)
train <- sample(nrow(Boston), nrow(Boston) * 2 / 3)
test <- setdiff(seq_len(nrow(Boston)), train)
hist(log(Boston$crim))
```



b. Propose a model (or set of models) that seem to perform well on this data set, and justify your answer. Make sure that you are evaluating model performance using validation set error, cross-validation, or some other reasonable alternative, as opposed to using training error.

Ans:

- We will try to fit models to `log(Boston$crim)` which is closer to a normal distribution.

```
fit <- lm(log(crim) ~ ., data = Boston[train, ])
mean((predict(fit, Boston[test, ]) - log(Boston$crim[test]))^2)
```

```
## [1] 0.66578
```

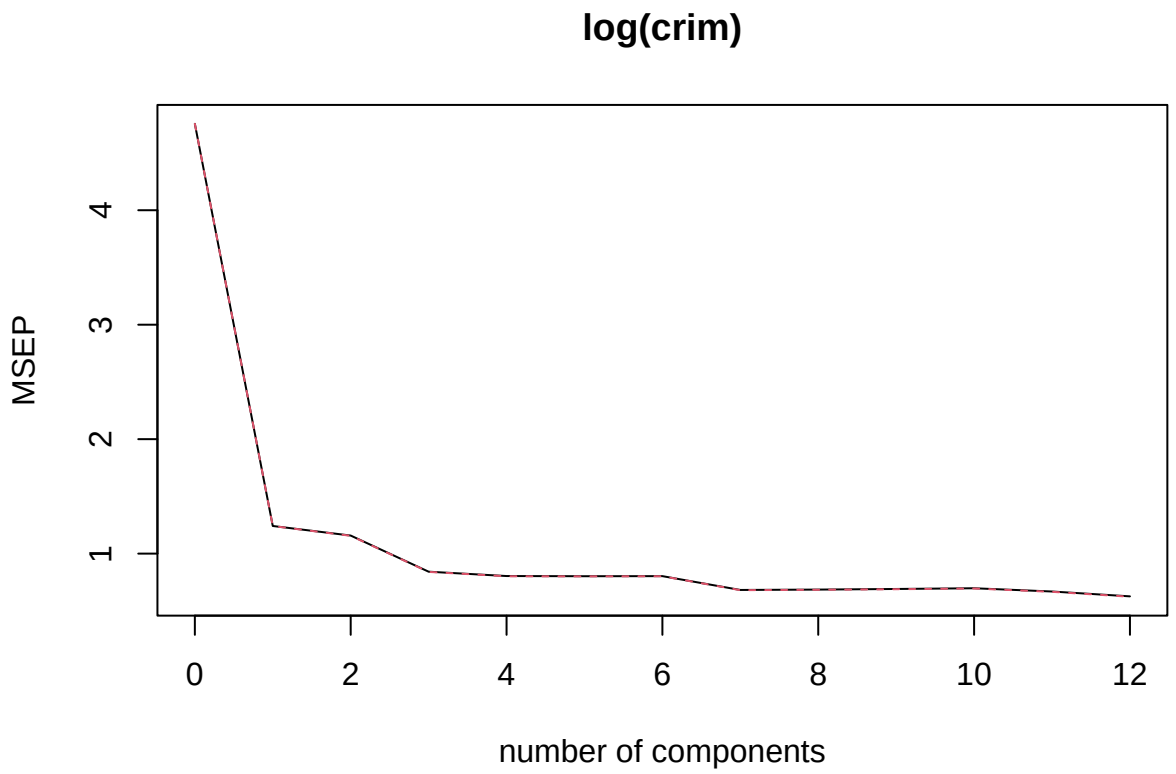
```
mm <- model.matrix(log(crim) ~ ., data = Boston[train, ])
fit2 <- cv.glmnet(mm, log(Boston$crim[train]), alpha = 0)
p <- predict(fit2, model.matrix(log(crim) ~ ., data = Boston[test, ]), s = fit2$lambda.min)
mean((p - log(Boston$crim[test]))^2)
```

```
## [1] 0.6511807
```

```
mm <- model.matrix(log(crim) ~ ., data = Boston[train, ])
fit3 <- cv.glmnet(mm, log(Boston$crim[train]), alpha = 1)
p <- predict(fit3, model.matrix(log(crim) ~ ., data = Boston[test, ]), s = fit3$lambda.min)
mean((p - log(Boston$crim[test]))^2)
```

```
## [1] 0.6494337
```

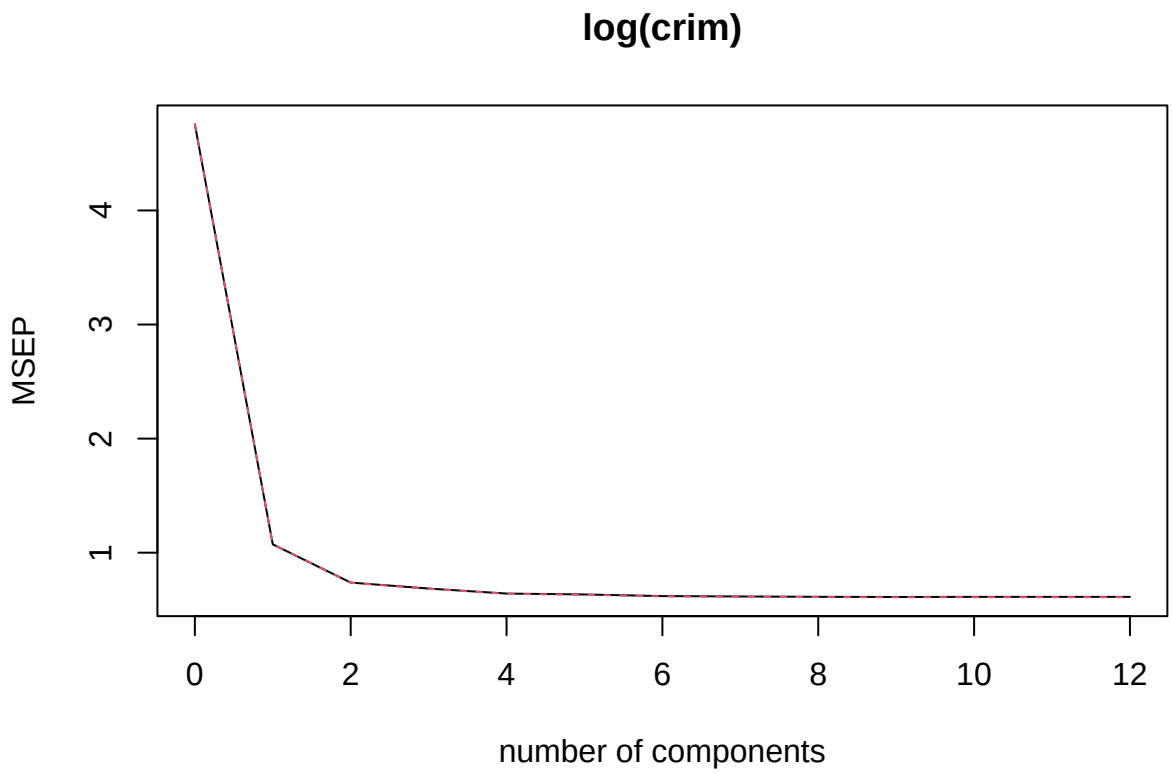
```
fit4 <- pcr(log(crim) ~ ., data = Boston[train, ], scale = TRUE, validation = "CV")
validationplot(fit4, val.type = "MSEP")
```




```
p <- predict(fit4, Boston[test, ], ncomp = 8)
mean((p - log(Boston$crim[test]))^2)
```

```
## [1] 0.6561043
```

```
fit5 <- plsr(log(crim) ~ ., data = Boston[train, ], scale = TRUE, validation = "CV")
validationplot(fit5, val.type = "MSEP")
```




```
p <- predict(fit5, Boston[test, ], ncomp = 6)
mean((p - log(Boston$crim[test]))^2)
```

```
## [1] 0.6773353
```

- In this case lasso (`alpha = 1`) seems to perform very slightly better than un-penalized regression. Some coefficients have been dropped:

```
coef(fit3, s = fit3$lambda.min)
```

```
## 14 x 1 sparse Matrix of class "dgCMatrix"
##                s=0.01483558
## (Intercept) -4.713172675
## (Intercept)  .
## zn          -0.011043739
## indus        0.022515402
## chas         .
## nox          3.856157215
## rm           .
## age          0.004210529
## dis          .
## rad          0.145604750
## tax          .
## ptratio     -0.031787696
```

```
## lstat        0.036112321
## medv         0.004304181
```

c. Does your chosen model involve all of the features in the data set? Why or why not?

Ans:

- Not all features are included due to the lasso penalization.